

Teoretická informatika

Obor C, 3. ročník

David Weber

SPŠE JEČNÁ

Poslední aktualizace: 26. června 2024

Obsah

Předmluva

Kapitola 1

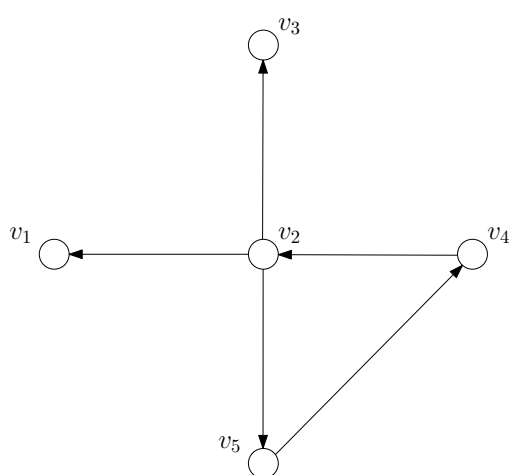
Grafové algoritmy

1.1 Grafy a jejich reprezentace

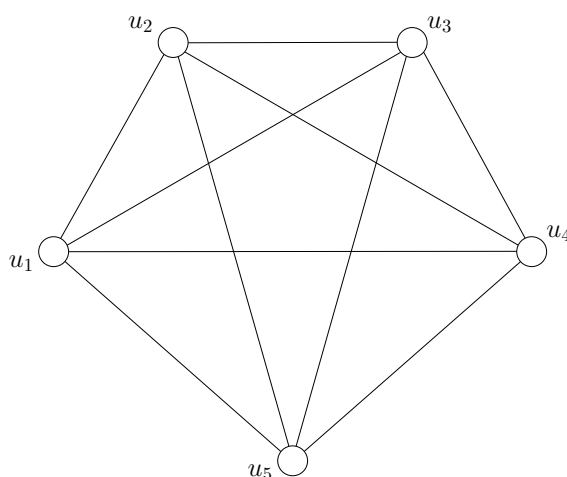
Grafy jsou základním stavebním kamenem pro mnoho úloh informatice. Lze jimi obecně reprezentovat mnoho situací, v nichž jsou klíčové určité prvky a jejich vzájemné vztahy. Např. propojení zařízení (router, switch, hub či koncový uživatel) v počítačové síti, schéma zapojení elektrického obvodu, mapa města nebo např. v chemii vazby mezi atomy. Jejich matematická podstata je však velice jednoduchá – *vrcholy* propojené *hranami*.

Definice 1.1.1 (Graf). Grafem G nazveme uspořádanou dvojici (V, E) , kde V je množina *vrcholů* (nebo také *uzlů*) a E množina *hran*, přičemž pokud

- $E \subseteq \{\{u, v\} \mid u, v \in V\}$, pak G nazýváme *neorientovaným* grafem (tj. po hraně lze pohybovat v obou směrech).
- $E \subseteq \{(u, v) \mid u, v \in V\}$, pak G nazýváme *orientovaným* grafem (tj. po hranách se lze pohybovat pouze v jednom směru).



(a) Příklad orientovaného grafu.

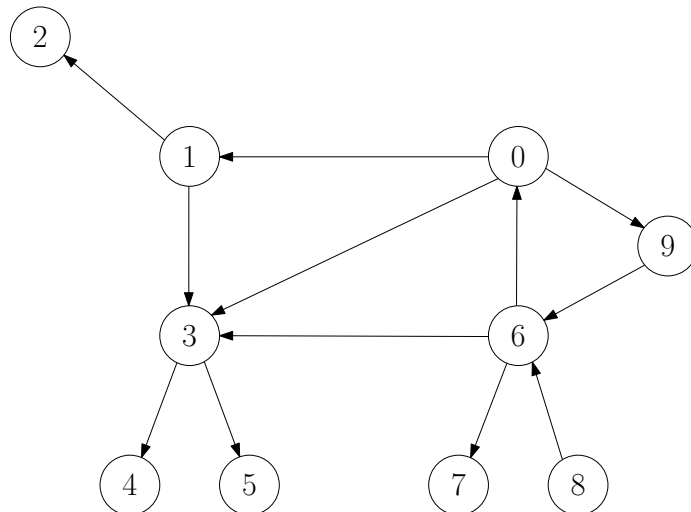


(b) Příklad neorientovaného grafu.

Podstatnou otázkou u grafů jsou způsoby jejich reprezentace, neboť při programování různé způsoby různě efektivní a je dobré si mezi nimi umět vybrat. Pracujme tedy s obecným grafem $G = (V, E)$, přičemž vrcholy grafu si pro jednoduchost označíme přirozenými čísly $1, 2, \dots, n$, tzn. $V = \{1, 2, \dots, n\}$.

- **Matice sousednosti.** První způsob, jak graf reprezentovat, je pomocí matice. V tomto případě graf G popisuje matice $n \times n$, kde na pozici ij je 1, pokud z vrcholu i do vrcholu j vede hrana, jinak 0.

Příklad 1.1.2 (Prásátkový graf). Orientovaný graf G (viz obrázek ??) a jeho matice sousednosti A .



Obrázek 1.2: Prásátkový graf.

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- **Matice incidence.** Řádky jsou indexovány vrcholy a sloupce jednotlivými hranami. Na místě ij je 1, pokud hrana vede *do* vrcholu i , nebo -1 , pokud z něj vychází, jinak 0. Pro neorientované grafy píšeme vždy 1, pokud daná hrana vede *do/z* vrcholu.

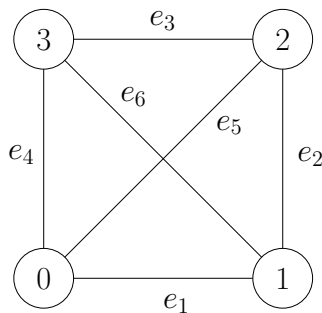
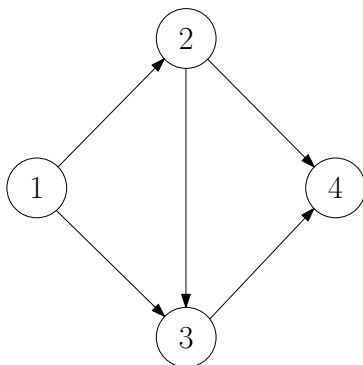
Příklad 1.1.3 (Úplný graf K_4). Graf G má 4 vrcholy a celkem 6 hran (viz obrázek ??) a její matice incidence I . Řádky představují (shora dolů) čísla vrcholů 0, 1, 2, 3 a sloupce (zleva doprava) jednotlivé hrany $e_1, \dots, e_6$.

$$I = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

- **Seznam sousedů.** Jedná se pravděpodobně o nejpoužívanější variantu reprezentace grafů. Pro každý vrchol $v \in V$ si jednoduše pamatujeme seznam¹ $S[v]$ všech vrcholů, do kterých přímo vede hrana z vrcholu v .

Příklad 1.1.4. Graf G s vrcholy $1, \dots, 4$ a jejich seznamy sousedů S (viz obrázek ??).

¹Formálně vzato se spíše jedná množinu.

Obrázek 1.3: Úplný graf K_4 .

$$\begin{aligned} S[1] &= \{2, 3\} & S[3] &= \{4\} \\ S[2] &= \{3, 4\} & S[4] &= \emptyset \end{aligned}$$

Obrázek 1.4: Graf G a seznamy sousedů pro jeho každý vrchol.

Tím jsme si ukázali základní možnosti reprezentace grafů, avšak zatím jsme si je mezi sebou nijak neporovnali. Pojďme se tedy podívat na paměťové nároky daných variant. Označme si počet vrcholů grafu $|V| = n$ a celkový počet hran $|E| = m$.

- *Matice sousednosti* spotřebuje paměť $\mathcal{O}(n^2)$, neboť potřebujeme uložit matici velikosti $n \times n$. Je tak vhodná především pro husté grafy, tedy grafy s velkým počtem hran.
- *Matice incidence* využije $\mathcal{O}(nm)$ paměťových buněk. Tento způsob reprezentace je vhodný především pro matematickou práci s grafy, avšak při programování se moc nevyužívá, neboť obsahuje spoustu nadbytečných informací.
- *Reprezentace pomocí seznamů sousedů* je obecně hodně využívána. Řadově spotřebuje $\mathcal{O}(n + m)$ paměti² a tak dobrou alternativou oproti matici sousednosti v případech, kdy naopak daný graf je řídký, tedy neobsahuje příliš mnoho hran (např. strom).

1.2 Stromy

Stromy tvoří speciální oblast v teorii grafů. V informatice se jedná silně dominující typ grafu a bylo by proto dobré mu věnovat zvláštní sekci, kde si připomeneme některé základní typy.

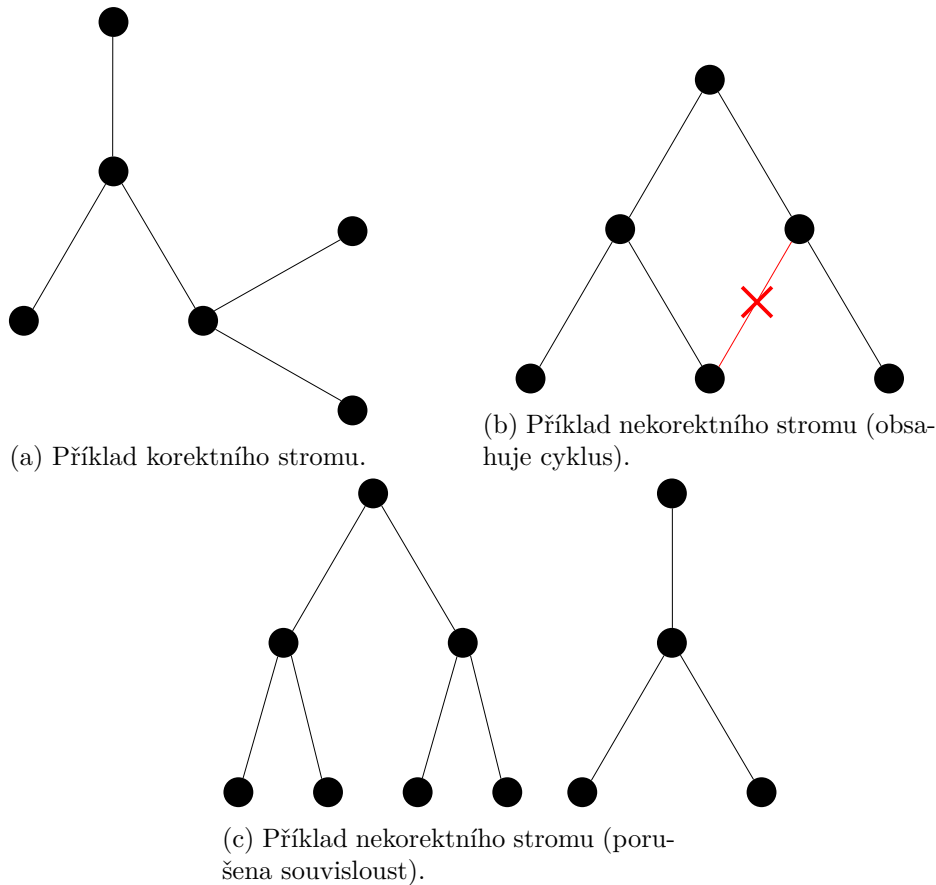
Definice 1.2.1 (Strom). Graf $G = (V, E)$ je strom, pokud

- (i) je souvislý³
- (ii) a neobsahuje kružnici.

²Odvození zatím necháme stranou, avšak jednoduchým důvodem je fakt, že si musíme v paměti uchovat celkem n vrcholů a pro každý z nich si musíme pamatovat všechny hrany, které z nich vychází. Blíže se na tento argument podíváme v odvození časové složitosti algoritmu BFS, viz ??.

³Mezi každou dvojicí vrcholů vede cesta.

Je dobré si uvědomit, že mezi každou dvojicí vrcholů ve stromě vede **právě jedna** cesta. Zároveň z definice stromu také plyne, že odebráním libovolné hrany se již nutně poruší souvislost (minimálně přestane exstovat cesta mezi koncovými vrcholy).



Poslední příklad ?? sice není z definice strom, avšak čtenář by mohl namítnout, že jednotlivé “části” daného grafu tvoří stromy. Tomuto typu grafů se říká (celkem pochopitelně) *lesy*, neboť jejich *komponenty souvislosti*⁴ tvoří stromy (viz obrázek ??).

Definice 1.2.2 (Les). Graf $G = (V, E)$ je les, pokud každá jeho komponenta souvislosti je strom.

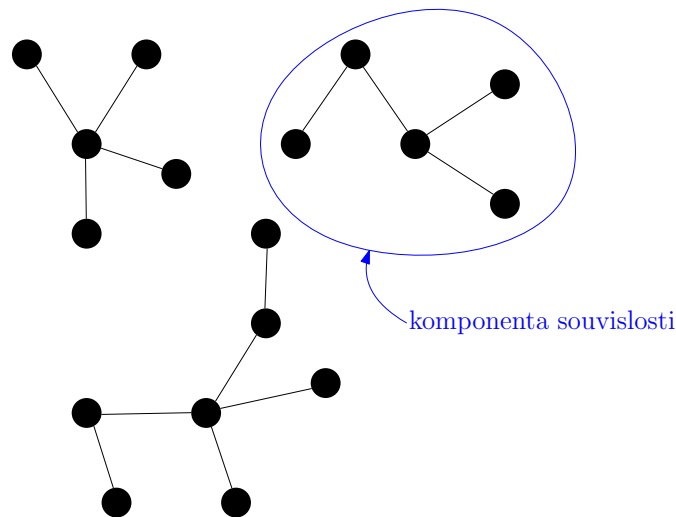
Ekvivalentně můžeme les definovat jako *sjednocení stromů*. Jako poslední si uvedme speciální typ stromu, se kterým budeme dále nejčastěji pracovat, a to tzv. *binární strom*. Blíže se s ním setkáme v sekci ??, kdy budeme rozebírat tzv. *binární haldu*.

Definice 1.2.3 (Binární strom). Strom $T = (V, E)$ nazveme binární, pokud

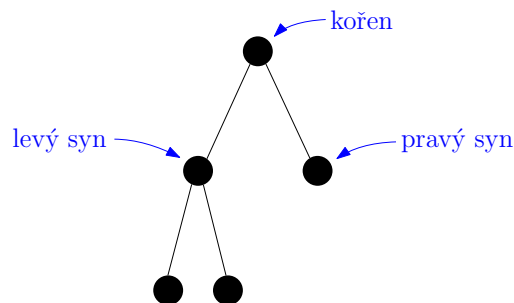
- (i) je zakořeněný
- (ii) a každý vrchol má nejvýše dva syny.

Jeden z vrcholů stromu T je označen jako *kořen*. Typicky jej zakreslujeme nahoru nad všechny ostatní vrcholy a syny každého vrcholu umísťujeme vždy níže oba na stejnou úroveň (viz obrázek ??).

⁴Intuitivně se jedná o část grafu, kde mezi každými dvěma vrcholy vede cesta. Formální zavedení termínu se provádí přes *binární relaci*. Pro zájemce: [https://en.wikipedia.org/wiki/Component_\(graph_theory\)](https://en.wikipedia.org/wiki/Component_(graph_theory)).



Obrázek 1.6: Příklad lesa



Obrázek 1.7: Příklad binárního stromu

1.3 Prohledávání do šířky

Jednou ze základních úloh je procházení grafu z určitého vrcholu a zjištění dosažitelnosti ostatních vrcholů. Nejjednodušším algoritmem v tomto ohledu je tzv. *prohledávání do šířky* (angl. *breadth-first search*, zkráceně BFS). Jeho základní princip spočívá v postupném objevování následníků již nalezených vrcholů. Na počátku dostaneme graf $G = (V, E)$ a nějaký počáteční vrchol $v_0 \in V$. Postupně objevíme všechny sousedy vrcholu v_0 , poté všechny sousedy těchto nalezených sousedů, atd. Na BFS lze nahlížet tak, že do počátečního vrcholu nalijeme vodu a sledujeme, jak postupuje vzniklá vlna.

Pro každý vrchol si budeme uchovávat jeho *stav*.

- *Nenalezený* – vrchol jsme ještě během výpočtu neviděli.
- *Otevřený* – vrchol jsme viděli, ale ještě nejsme neprozkoumali všechny jeho sousedy.
- *Uzavřený* – vrchol jsme prozkoumali společně se všemi jeho sousedy a dál se jím již netřeba zabývat.

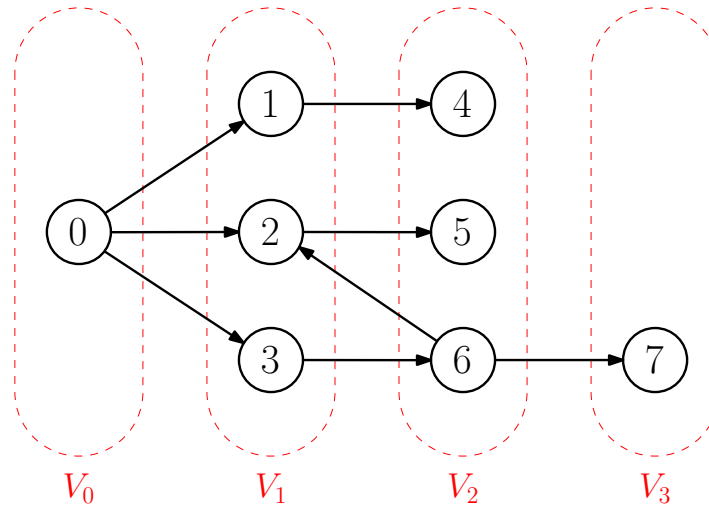
Na počátku začneme s jedním otevřeným vrcholem a to v_0 (zde začínáme). Po prozkoumání všech sousedních vrcholů se jejich stav změní na uzavřený a počáteční vrchol v_0 se uzavře. Obdobně pokračujeme pro nově otevřené vrcholy. Pokud by náhodou mezi dvojicí otevřených vrcholů existovala hrana, pak si sousedního vrcholu všimnat nebudeme, neboť byl již otevřen. Pro každý vrchol se ještě dodatečně můžeme uchovávat informaci, jak daleko se nachází od v_0 , co do počtu hran ležících na cestě.

BFS rozděluje vrcholy do vrstev podle toho, v jaké vzdálenosti od počátečního vrcholu se nachází (viz obrázek ??). Nejdříve jsou prozkoumány vrcholy ve vzdálenosti 0 (tj. pouze v_0), poté ve vzdálenostech 1,

Algoritmus 1.3.1: BFS (prohledávání do šířky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
 // Inicializace
 1 **foreach** vrchol $v \in V$ **do**
 2 $stav(v) \leftarrow \text{nenalezený}$
 3 $D(v) \leftarrow \infty$
 4 $stav(v_0) \leftarrow \text{otevřený}$
 5 $D(v_0) \leftarrow 0$
 6 Založ frontu Q a přidej do ní vrchol v_0
 7 **while** fronta Q je neprázdná **do**
 8 $v \leftarrow$ první vrchol ve frontě Q , který z ní odebereme
 9 // Prozkoumáváme všechny dosud neobjevené sousedy
 10 **foreach** sousední vrchol w vrcholu v **do**
 11 **if** $stav(w) = \text{nenalezený}$ **then**
 12 $stav(w) \leftarrow \text{otevřený}$
 13 $D(w) \leftarrow D(v) + 1$
 14 Přidej w do fronty Q
 15 $stav(v) \leftarrow \text{uzavřený}$
Výstup: Seznam vzdáleností D

2, ... Z toho vyplývá, že kdykoliv při otvírání libovolného vrcholu v nastavujeme hodnotu $D(v)$, bude tato hodnota vždy odpovídat délce nejkratší cesty z v_0 do v . Tato hodnota bude však nastavena pouze u těch vrcholů, které jsou z v_0 dosažitelné (ostatní vrcholy zůstanou ve stavu nenalezený).



Obrázek 1.8: Vrcholy rozdělené do vrstev podle průběhu BFS.

Zbývá prozkoumat časovou a paměťovou složitost BFS. Označme si počet vrcholů grafu G na vstupu n a počet jeho hran m .

Věta 1.3.1 (Složitost BFS). Algoritmus BFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Inicializace potrvá $\mathcal{O}(n)$, neboť cyklus iteruje přes všechny vrcholy. Vnější cyklus provede maximálně n iterací, protože každý z vrcholů uzavřeme nejvýše jednou, tj. $\mathcal{O}(n)$.

S vnitřním cyklem je to trochu složitější, protože jeho počet iterací závisí na tom, který z vrcholů

otevíráme (resp. na počtu jeho sousedů). To znamená, že pokud si označíme d_i počet sousedů vrcholu i , pak celkový počet iterací vnitřního cyklu přes všechny vrcholy bude $\sum_i d_i$. Lze si ovšem všimnout jedné užitečné věci. Pokaždé, když prozkoumáváme sousední vrchol w nějakého vrcholu v , mohou nastat dva případy podle toho, jestli je G orientovaný graf, nebo neorientovaný.

- (i) Graf G je neorientovaný. Pak hranu, která spojuje v a w prozkoumáme právě *dvakrát* (jednou z vrcholu v a podruhé z vrcholu w). Každá hrana se tak započítá dvakrát, tzn. vnitřní cyklus se celkově provede max $2m$ -krát (po všech iteracích vnějšího cyklu).
- (ii) Graf G je orientovaný. Pak se hrana započítá pouze jednou, a to z vrcholu, z něhož vede. Celkově se vnitřní cyklus provede m -krát.

V prvním případě tak pro časovou složitost bude platit

$$\mathcal{O}(n + \sum_i d_i) = \mathcal{O}(n + 2m) \stackrel{*}{=} \mathcal{O}(n + m).$$

(V kroku označeném $*$ zanedbáváme konstantu, neboť pracujeme s $\mathcal{O}$ -notací.)

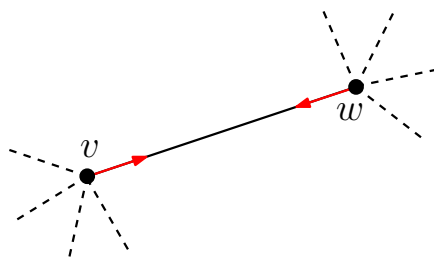
V druhém případě dojdeme ke stejnému výsledku, akorát zmíněná suma bude, kvůli orientaci hran, rovna přesně počtu hran, tj.

$$\mathcal{O}(n + \sum_i d_i) = \mathcal{O}(n + m).$$

Nyní k prostorové složitosti algoritmu. Na reprezentaci grafu (např. pomocí seznamu sousedů) spotřebujeme paměť $\mathcal{O}(n + m)$ (n vrcholů, m hran)⁵. Dále si uchováváme vrcholy ve frontě, v níž může být v jednu chvíli maximálně n vrcholů, tj. $\mathcal{O}(n)$, a k tomu máme seznam *stav*, kde je uloženo $\mathcal{O}(n)$ vrcholů. Celkově spotřebujeme paměti

$$\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m).$$

□



Obrázek 1.9: Znázornění situace v důkazu části ?? věty ??.

1.4 Prohledávání do hloubky

Na podobném přístupu, jako BFS, je založeno tzv. *prohledávání do hloubky* (anglicky *depth-first search*, zkráceně DFS). Vrcholy však tentokrát budeme zpracovávat rekurzivně. Pokaždé, když budeme otevírat nový vrchol, se rekurzivně zavoláme všechny jeho sousední vrcholy, u nichž opakujeme stejnou proceduru. Po prozkoumání všech sousedů daný vrchol uzavřeme. Stejně jako BFS si tak budeme uchovávat pole *stavů* pro jednotlivé vrcholy.

⁵Resp. každá hrana je v neorientovaných grafech započítána dvakrát, protože každý vrchol obsahuje v seznamu svých sousedů vždy „protějščí“ vrchol (stejný argument, jako u odvozování časové složitosti BFS).

Algoritmus 1.4.1: DFS (prohledávání do hloubky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
 // Inicializace
 1 **foreach** vrchol $v \in V$ **do**
 2 $stav(v) \leftarrow \text{nenalezený}$
 3 Zavolej DFS2(v_0)
 // Rekurzivní volání na sousední vrcholy
 4 **Funkce** DFS2(vrchol $v \in V$)
 5 $stav(v) \leftarrow \text{otevřený}$
 6 **foreach** sousední vrchol w vrcholu v **do**
 7 **if** $stav(w) = \text{nenalezený}$ **then**
 8 Zavolej DFS2(w)
 9 $stav(v) \leftarrow \text{uzavřený}$

Věta 1.4.1 (Složitost DFS). Algoritmus DFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Algoritmus DFS se oproti BFS liší pouze v pořadí, v jakém pořadí dosažitelné vrcholy zpracovává, to však nemá na časovou složitost žádný vliv. Argument pro její odvození je tak stejný jako u BFS, viz věta ?? v minulé sekci.

V paměti si musíme uchovávat reprezentaci grafu, to zabere $\mathcal{O}(n + m)$ paměti (opět např. pomocí seznamu sousedů) a máme seznam $stav$ s n prvky (vrcholy). Zároveň si při volání DFS2 musíme na zásobník rekurze ukládat jednotlivé *aktivační záznamy*. Protože vrcholů je v grafu n , pak na zásobníku rekurze bude v jednu chvíli maximálně n záznamů, tj. $\mathcal{O}(n)$. Celkově spotřebujeme $\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m)$ paměti. $\square$



Je dobré si uvědomit, že byť algoritmy BFS a DFS mají stejnou časovou a prostorovou složitost, nelze zcela rovnocenně použít na stejné typy úloh. Např. BFS se hodí pro hledání nejkratší cesty *v neohodnoceném grafu* (pro ohodnocené grafy se používá např. Dijkstrův algoritmus, viz sekce ??), kdežto DFS se více hodí na prohledávání stavového prostoru, neboť typicky nezabere tolik paměti.



Cesta, kterou se DFS dostane do libovolného vrcholu v , nemusí být nutně nejkratší (silně závisí na pořadí, v jakém procházíme sousedy jednotlivých vrcholů).

1.5 Binární halda

Uvedme motivační příklad na úvod. Mějme seznam čísel, z něhož chceme (pokud možno co nejrychleji) vybrat minimální/maximální hodnotu. Pro maximum bychom sestavili následující jednoduchý algoritmus.

Časová složitost bude zjevně $\mathcal{O}(n)$, neboť algoritmus prochází všech n prvků. Takovou úlohu lze však řešit rychleji, pokud si prvky vhodně uspořádáme. K tomu můžeme použít tzv. *haldu*.

Algoritmus 1.5.1: Vyhledání maximální hodnoty v seznamu (MAX)**Vstup:** Seznam čísel $x_1, x_2, \dots, x_n$

```

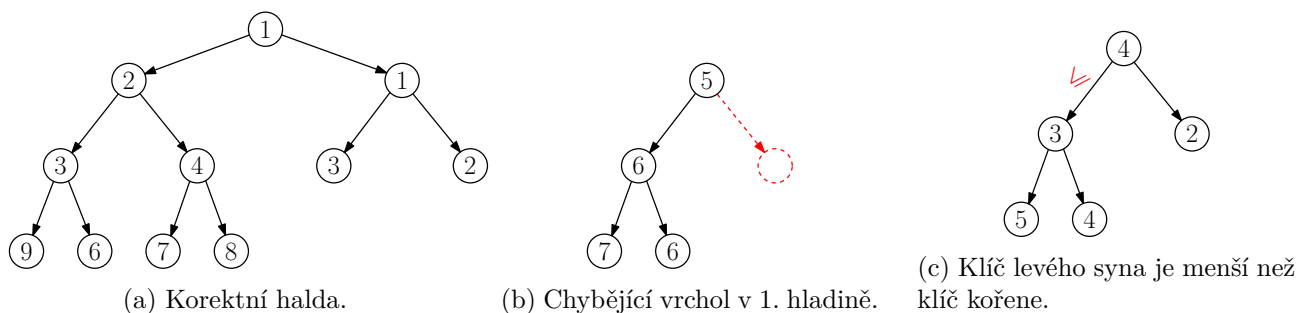
1  $m \leftarrow x_1$ 
2 for  $i = 1, 2, \dots, x_n$  do
3   if  $x_i > m$  then
4      $m \leftarrow x_i$ 
5 return  $m$ 

```

Výstup: Maximální hodnota seznamu m

Definice 1.5.1 (Minimová binární halda). Minimová binární halda je datová struktura tvaru binárního stromu, kde v každém vrcholu je uložena *právě jedna* hodnota (tzv. *klíč*, pro vrchol v budeme značit jeho klíč $k(v)$) a navíc platí:

- (i) každá hladina je *plně obsazena*, kromě poslední, přičemž hladiny jsou zaplněny *zleva*
- (ii) a je-li v libovolný vrchol a s jeho syn, pak $k(v) \leq k(s)$.



Obrázek 1.10: Příklady korektních a nekorektních hald.

Zde se nyní na chvíli pozastavme. Podmínka ?? v definici binární haldy ?? má za následek totiž velmi příjemnou vlastnost, když se podíváme, kde se v haldě nachází minimum. Pokud se budeme pohybovat od kořene směrem „dolů“, hodnoty ve vrcholech se budou pouze zvětšovat, neboť klíče synů musí mít vždy stejnou nebo větší hodnotu než klíč v rodiči. Minimum se tak vždy nachází *v kořeni stromu*, což znamená, že zjištění minima tak můžeme provést v *konstantním čase* $\mathcal{O}(1)$.



Pokud bychom chtěli naopak *maximovou binární haldu*, bude definice ?? vypadat obdobně, akorát v podmínce ?? bude obrácená nerovnost, tj. $k(v) \geq k(s)$ (klíč v rodiči má vždy stejnou nebo vyšší hodnotu než klíče v jeho synech). Maximum se bude opět nacházet v kořeni haldy.

Je však více operací, které bychom rádi s haldou prováděli. Vypišme si všechny, které nás budou zajímat.

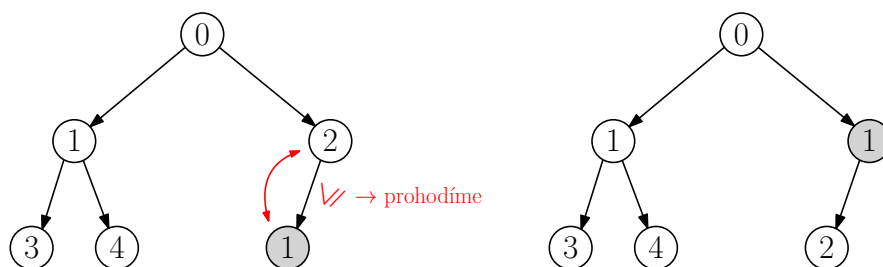
- INSERT(x) – *vložení* nového vrcholu s klíčem x do haldy.
- MIN() – *nalezení* vrcholu s nejmenším klíčem (už jsme zmínili).
- EXTRACTMIN() – *odebrání* vrcholu s nejmenším klíčem.
- INCREASE(v) – *zvýšení* hodnoty klíče ve zvoleném vrcholu v .
- DECREASE(v) – *snížení* hodnoty klíče ve zvoleném vrcholu v .

Podívejme se postupně na jednotlivé operace a jejich realizaci. S dovolením si zde nyní odpustíme zápis

pomocí pseudokódu a pro jednoduchost si dané operace pouze slovně vysvětlíme. Pro následné odvození časové složitosti nám to bude stačit.

1.5.1 Vkládání nového vrcholu

Při vkládání nového vrcholu (INSERT) se nejdříve potřebujeme vypořádat s tím, kam vrchol v haldě umístíme. S tím nám ale pomůže podmínka ?? v definici binární haldy. Všechny hladiny stromu jsou vždy plně obsazené, kromě poslední, která se všechny vrcholy nachází vlevo. To znamená, že nový vrchol můžeme umístit na první volnou pozici vlevo na poslední hladině. To však nemůžeme provést zcela beztestně, neboť nemáme nijak zaručeno, že bude splněna podmínka ?. Klíč v novém vrcholu může mít hodnotu *ostře menší* než jeho rodič. Haldu „opravíme“ postupným prohazováním vrcholu s jeho rodičem, dokud nebude podmínka splněna (viz obrázek ??). Postup můžeme zapsat zkráceně ve třech



Obrázek 1.11: Vkládání nového vrcholu do haldy.

bodech takto:

- Vložíme nový vrchol s klíčem x na první volnou pozici zleva na poslední hladině.
- pokud je klíč rodiče větší než x , prohodíme vrcholy (resp. jejich klíče)⁶.
- Opakujeme druhý krok.

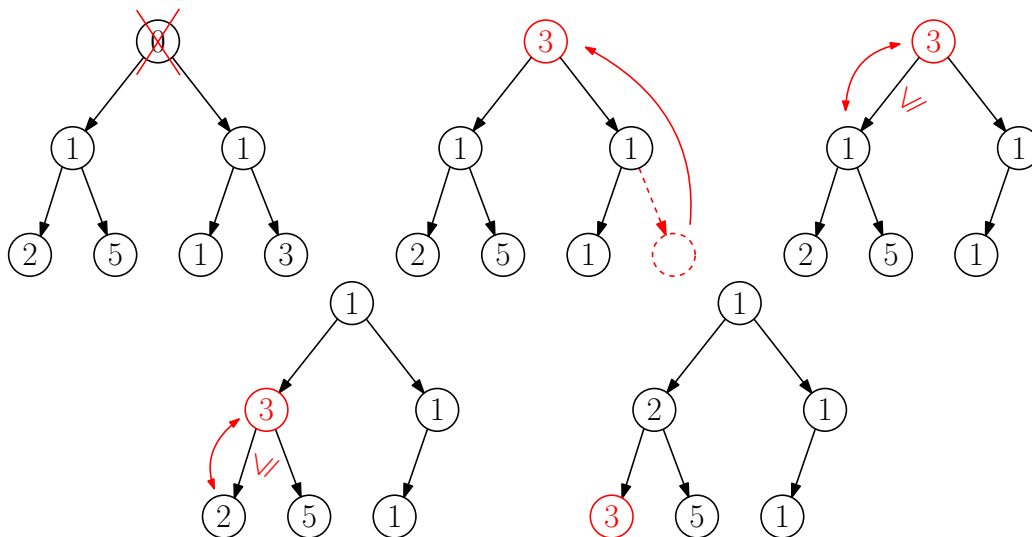
1.5.2 Odstranění minima

Co se týče získání minimálního klíče v haldě, jedná se o velmi triviální operaci. Zkrátka „přečteme“ klíč v kořeni a jsme hotovi. Ovšem pokud bychom chtěli minimum (EXTRACTMIN) ze stromu odstranit, bude to již o něco složitější. Kořen ze stromu nemůžeme „jen tak“ smazat, jeho pozici musí zastoupit jiný vrchol. Abychom neporušili vlastnosti haldy, nahradíme odstraněný kořen *nejlevějším vrcholem na poslední hladině*. Tím jsme jistě neporušili vlastnost ??, neboť poslední hladina, jako jediná, nemusí být plně obsazená.

Nyní však (stejně jako u vkládání vrcholu) nastává problém s druhou podmínkou, neboť touto operací jsme ji mohli porušit. Provedeme obdobný proces, jako při vkládání vrcholu do haldy, ale vrchol nyní bude „probublávat“ směrem dolů. Podíváme se na syny daného vrcholu a pokud klíč některého z nich je menší než klíč nového kořene, pak jej s ním prohodíme (v případě, že klíče obou synů jsou menší, než klíč kořene, prohodíme s menším z nich). V bodech můžeme zapsat celou operaci následovně:

- Smažeme kořen a nahradíme jej nejlevějším vrcholem na poslední hladině.
- Tento vrchol (resp. jeho klíč) porovnáme s jeho syny a případně prohodíme s tím, jehož klíč je menší.

⁶Vrchol v podstatě takto „probublá“ do správné pozice ve stromě (příp. až do pozice kořene).



Obrázek 1.12: Odebírání vrcholu (kořene) s minimálním klíčem.

- Opakujeme druhý krok.

Operacím, které jsme prezentovali při vkládání vrcholu, resp. odebírání minima z haldy, kdy vrchol „probublával“ nahoru, resp. dolů, se někdy nazývají BUBBLEUP a BUBBLEDOWN. Ještě se nám budou hodit u operací INCREASE a DECREASE.

1.5.3 Zvýšení a snížení hodnoty klíče ve vrcholu

Poslední dvojice operací, která se nám bude hodit, je zvýšení (INCREASE) a snížení (DECREASE) hodnoty klíče ve vybraném vrcholu. K tomu potřebujeme akorát vyřešit, jak budeme k vrcholům přistupovat. Podle klíče vyhledávat neumíme, ale můžeme si zavést např. pomocný slovník, který budeme „adresovat“ pomocí příslušného vrcholu a vrácenou hodnotou bude klíč ve vrcholu. Takto spotřebujeme navíc pouze $\mathcal{O}(n)$ paměti, kde n je počet vrcholů v haldě.

Realizace samotných operací je již jednoduchá. Pokud provedeme INCREASE klíče v určitém vrcholu, pak z důvodu, že klíče v synech vrcholu jsou stejné nebo větší, se halda může pokazit pouze „směrem dolů“. Tedy provedeme nám již známé prohazování vrcholu s jeho syny, dokud nebude halda opravena (BUBBLEDOWN), stejně jako v případě odebírání kořene (EXTRACTMIN).

Operaci DECREASE provedeme zcela stejně, akorát nyní se halda bude kazit „směrem nahoru“, takže provedeme BUBBLEUP, jako při vkládání nového vrcholu (INSERT).



V případě *maximové binární haldy* by operace vypadaly zcela stejně, akorát při operaci INCREASE budeme opravovat haldu směrem nahoru a u DECREASE směrem dolů.

1.5.4 Časové složitosti operací

Pojďme si nyní rozebrat časové složitosti zmíněných operací INSERT, MIN, EXTRACTMIN, INCREASE a DECREASE a podívat se, jak moc jsme si pomohli oproti práci s polem (seznamem). Pokud se pozorně podíváme na operace popsané výše, je zjevné, že nejvíce bude naše operace zpomalovat ono „probublávání“ vrcholu při opravování haldy (vyjma operace MIN, kde jen přečteme klíč v kořeni), tj. BUBBLEUP

a BUBBLEDOWN. Na nich bude celková časová složitost záviset.



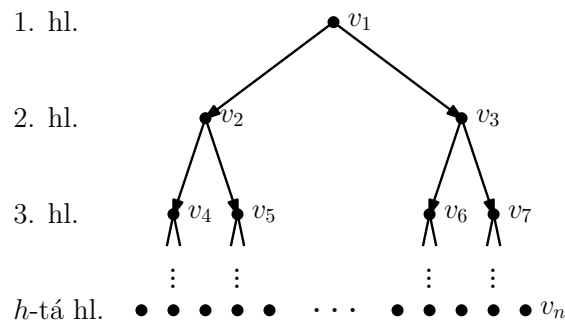
Všimněme si, že po každém prohození vrcholů při BUBBLEUP nebo BUBBLEDOWN se vrchol posune o jednu hladinu výše/níže.

Víme tak, že při opravování haldy se vrchol může posunout *maximálně o počet hladin*, který má daná halda. Časová složitost daných operací bude tak závislá na jejich počtu. Tím jsme se dostali o trochu blíže řešení, ale rádi bychom věděli, jak se bude dobře trvání daných operací měnit v závislosti na *počtu vrcholů v haldě*, nikoliv počtu hladin. Mezi těmito proměnnými však existuje hezká závislost.

Dejme tomu, že naše halda má n vrcholů v celkově h hladinách, přičemž poslední hladina je *plně obsazena* (viz obrázek ??). (Mohlo by se, v obecném případě, samozřejmě stát, že poslední hladina plně obsazena nebude, ale jedině, co se tím změní, bude, že mezi n a výsledným výrazem bude místo rovnosti nerovnost. S plnou hladinou se nám ale bude lépe počítat, protože při analýze časové složitost nás zajímá nejhorší možný případ.) Pokud budeme mít haldu s jednou hladinou, bude mít pouze jeden vrchol, a to kořen. S dvěma hladinami budou již 3 vrcholy v haldě. S třemi hladinami jich bude 7, atd (viz tabulka ??).

Počet hladin h	1	2	3	4	5	6
Počet vrcholů n	1	3	7	15	31	63

Tabulka 1.1: Vztah mezi počtem hladin h a počtem vrcholů n v plně obsazené haldě.



Obrázek 1.13: Odebírání vrcholu (kořene) s minimálním klíčem.

Z tabulky ?? si můžeme všimnout, že počet vrcholů je vždy *mocnina dvojky snižená o jedna*. Výsledná závislost je tedy $n = 2^h - 1$. V případě, že by poslední hladina nebyla plně obsazena, by se rovnost změnila v nerovnost, tj. $n \leq 2^h - 1$. Tím máme již vše připravené k tomu, abychom odvodili časovou složitost daných operací.

Věta 1.5.2 (Složitost operací v binární haldě). Operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají časovou složitost $\mathcal{O}(\log n)$. Operace MIN má časovou složitost $\mathcal{O}(1)$.

Důkaz. V případě MIN je to jednoduché, zkrátka jen přečteme klíč v kořeni, což zjevně potrvá $\mathcal{O}(1)$. U zbývajících operací opravujeme haldu pomocí BUBBLEUP a BUBBLEDOWN. Předpokládejme, nastal nejhorší možný případ, tj. halda má h hladin a je plně obsazena (tzn. včetně poslední hladiny). Víme, že „probublání“ vrcholu haldou nahoru/dolů potrvá řádově $\mathcal{O}(h)$. My však víme, že $n = 2^h - 1$. Tedy zde bude stačit, když si vyjádříme h z rovnosti.

$$n = 2^h - 1 \iff 2^h = n + 1 \iff h = \log_2(n + 1) \stackrel{*}{=} \frac{1}{\log 2} \cdot \log(n + 1).$$

(V rovnosti $*$ jsme využili vzorec $\log_b a = \log a / \log b$.) Operace BUBBLEUP a BUBBLEDOWN tedy potrvají

$$\mathcal{O}(h) = \mathcal{O}\left(\frac{1}{\log 2} \cdot \log(n+1)\right) \stackrel{*}{=} \mathcal{O}(\log n).$$

(V $*$ je jednička v logaritmu zanedbatelná v rámci $\mathcal{O}$ -notace. Výraz $1/\log 2$ představuje konstantu a tudíž je též zanedbatelný.) Tedy operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají logaritmickou časovou složitost. $\square$

Zkusme si na závěr udělat menší porovnání oproti poli, se kterým jsme začínali (viz tabulka ??). Lo-

	INSERT	MIN	EXTRACTMIN	INCREASE	DECREASE
Pole (seznam)	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Binární halda	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Tabulka 1.2: Porovnání časových složitostí operací v haldě a v poli.

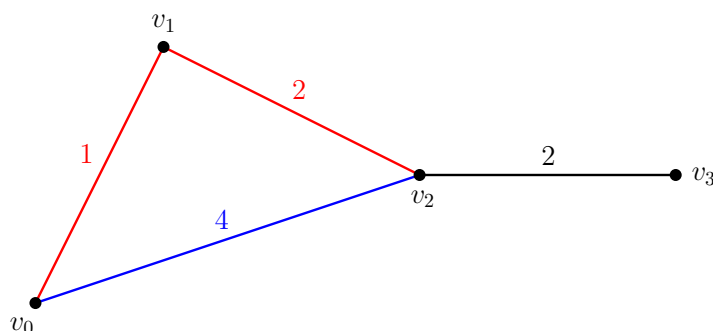
garitmická časová složitost je určitě lepší, než lineární, je však vidět, že jsme si zároveň pohoršili v případě vkládání a úpravy hodnot klíčů. Je tomu tak z důvodu, že pole má svou výhodu v téměř nulové režii, kdežto halda (kvůli své pevně definované struktuře) spotřebuje nějaký čas, aby byla uvedena do korektního stavu.

1.6 Dijkstrův algoritmus

Již jsme si ukázali, že v *nehodnoceném grafu* $G = (V, E)$ umíme relativně jednoduše najít délku nejkratší cesty do libovolného vrcholu z nějakého výchozího vrcholu $v_0 \in V$. Obvykle však hrany nemusí mít stejnou váhu (cenu). Např. cesty (vrcholy) mezi městy (hrany) mohou být v různém stavu a některé jsou tak lepší než jiné.



Zde ji narážíme na problém, neboť obecně platí, že když nalezneme vrchol přes dvojici různých hran, nemusí být nalezené cesty stejně dlouhé. Na obrázku ?? si lze všimnout, že cesta (v_0, v_1, v_2) je kratší než (v_0, v_2) , přestože má více hran.



Obrázek 1.14: Příklad ohodnoceného grafu.

Nabízí se varianta převést ohodnocený graf na neohodnocený tak, že rozdělíme hranu na takový počet (nehodnocených) hran, kolik činí její původní váha. V čistě teoretické rovině se jedná o funkční řešení, neboť na nově vzniklý graf již lze aplikovat např. BFS, které jsme popsali v sekci ??.

Ne každý graf však musí mít „malé“ váhy hran. Pokud bychom vzali např. graf, kde váhy hran jsou v řádech tisíců, bude pro

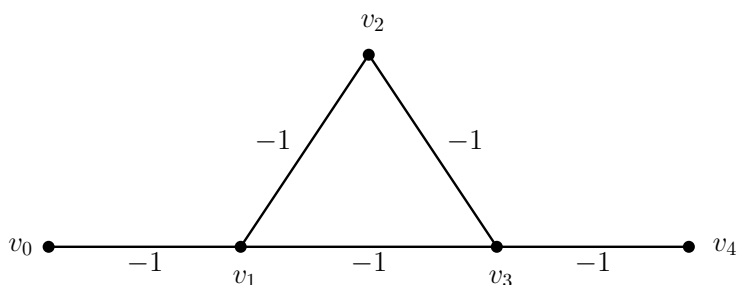
BFS potřeba provést zbytečně mnoho práce, neboť pro zpracování jedné ohodnocené hrany bude třeba provést tisíce iterací.

Jedním z nejznámějších algoritmů v tomto ohledu je tzv. *Dijkstrův algoritmus*, který popsal v roce 1958 holandský informatik *Edsger Wybe Dijkstra*.⁷ Podobně jako u BFS a DFS, i zde budeme postupně *otevírat* a *uzavírat* vrcholy, přičemž každý uzavřeme *nejvýše jednou*, avšak je třeba mít na paměti důležitou věc.



První nalezená cesta do libovolného vrcholu již nemusí být nutně nejkratší. Cestu do daného vrcholu bude třeba postupně vylepšovat.

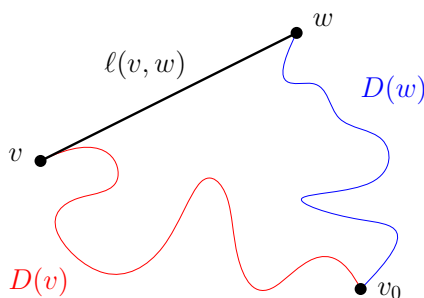
Zaměříme se na grafy, jejichž váhy hran jsou nezáporné. Záporně ohodnocené hrany mohou způsobovat problémy, neboť mezi určitou dvojicí vrcholů by již nemusela nutně existovat nejkratší cesta. Např. v obrázku ?? si lze všimnout, že mezi vrcholy v_0 a v_4 neexistuje nejkratší cesta konečné délky, neboť cyklus (v_1, v_3, v_2, v_1) lze projít libovolněkrát, přičemž každým průchodem se zkrátí o 3. Odpověď na délku nejkratší cesty mezi těmito vrcholy by tak musela být $-\infty$. Podobným grafům se tak chceme



Obrázek 1.15: Graf, kde mezi v_0 a v_4 neexistuje (konečná) nejkratší cesta.

vyhnout, neboť by naše úvahy pak již nemusely fungovat (cestu mezi vrcholy bychom mohli potenciálně do nekonečně vylepšovat a algoritmus by se tak nikdy nezastavil).

V dalších odstavcích budeme váhu hrany vedoucí z vrcholu u do vrcholu v značit $\ell(u, v)$.



Obrázek 1.16: Znázornění situace v průběhu Dijkstrova algoritmu.

1.6.1 Správnost Dijkstrova algoritmu

Nejprve je dobré si uvědomit, proč algoritmus vlastně funguje. Kdykoliv uzavíráme vrchol (tzn. vybereme vrchol s nejmenší hodnotou $D(v)$), pak se spoléháme na to, že $D(v)$ opravdu odpovídá v daný moment délce nejkratší cesty z v_0 do v . Ale proč to tak musí být?

⁷ Jedná se o holandské jméno, čteme „dajkstra“.

Algoritmus 1.6.1: DIJKSTRA

Vstup: Nezáporně ohodnocený graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

// Inicializace

```

1 foreach vrchol  $v \in V$  do
2    $stav(v) \leftarrow \text{nenalezený}$ 
3    $D(v) \leftarrow \infty$ 
4  $stav(v_0) \leftarrow \text{otevřený}$ 
5  $D(v_0) \leftarrow 0$ 
6 while existují otevřené vrcholy do
7   // Kandidát na nejkratší cestu do sousedních vrcholů
8    $v \leftarrow$  otevřený vrchol s nejmenším  $D$ 
9   foreach soused  $w \in V$  vrcholu  $v$  do
10    // Našli jsme lepší cestu
11    if  $D(v) + \ell(v, w) < D(w)$  then
12       $D(w) \leftarrow D(v) + \ell(v, w)$ 
13       $stav(w) \leftarrow \text{otevřený}$ 
14  $stav(v) \leftarrow \text{uzavřený}$ 

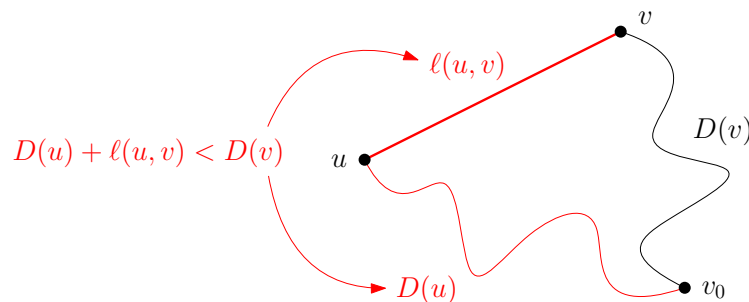
```

Výstup: Seznam vzdáleností D

Zkusme uvážit opačnou situaci, tzn. kdyby nastalo, že jsme vybrali nějaký vrchol v , jehož $D(v)$ neodpovídá délce nejkratší cesty. Pak by musel existovat nějaký sousední vrchol u vrcholu v , přes nějž vede nejkratší cesta, jejíž délka je $D(u) + \ell(u, v)$ (viz obrázek ??). To znamená, že $D(v) > D(u) + \ell(u, v)$. Jenže v takovou chvíli by si algoritmus nutně musel vybrat vrchol u místo vrcholu v , protože

$$D(v) > D(u) + \ell(u, v) \stackrel{*}{>} D(u).$$

V kroku označeném $*$ vycházíme právě z toho, že váhy hran jsou nezáporné, tj. $\ell(u, v) > 0$ (proto tento argument funguje). Tedy stručně řečeno⁸, pokud $D(v)$ neodpovídá délce nejkratší cesty, pak musí existovat jiný vrchol u s nižším $D(u)$.



Obrázek 1.17: Situace při výběru vrcholu v , kdy $D(v)$ neodpovídá délce nejkratší cesty.

1.6.2 Časová složitost Dijkstrova algoritmu

Zbývá nám ještě analyzovat časovou složitost. Pokud se zpětně podíváme na pseudokód algoritmu, lze si všimnout, že podstatnou roli bude hrát vybírání vrcholu s minimální hodnotou $D(v)$. Nabízí se tak otázka, jak danou operaci implementovat. První variantou je hodnoty D realizovat jako prostý *seznam* (popř. *pole*).

⁸Nebo spíš napsáno?

Věta 1.6.1 (Dijkstra se seznamem). Dijkstrův algoritmus, kde hodnoty D ukládáme do seznamu (pole), doběhne v čase $\mathcal{O}(n^2 + m)$.

Důkaz. • Inicializace zabere $\mathcal{O}(n)$ (máme n vrcholů).

- Každý vrchol uzavřeme opět nejvýše jednou, tzn. vnější cyklus se provede $\mathcal{O}(n)$ -krát.
- Hledání minimálního D zabere v rámci každé iterace čas $\mathcal{O}(n)$.
- Stejně jako u BFS a DFS, i zde každou hranu zkontroluji nejvýše dvakrát (záleží, zda graf G je orientovaný), tzn. $\mathcal{O}(2m) = \mathcal{O}(m)$.

Celkově tedy máme

$$\underbrace{\mathcal{O}(n)}_{\text{Init}} + \underbrace{\mathcal{O}(n) \cdot \mathcal{O}(n)}_{\text{Výběr minima}} + \mathcal{O}(m) = \mathcal{O}(n^2 + n + m) = \mathcal{O}(n^2 + m).$$

□

Kvadratická časová složitost není úplně zlá, ale můžeme si ještě trochu přilepšit. Pokud si vzpomeneme na binární haldy z oddílu ??, všimneme si v konečném důsledku, že při použití v Dijkstrově algoritmu se některé operace zrychlí.

Věta 1.6.2 (Dijkstra s haldou). Dijkstrův algoritmus, kde hodnoty D ukládáme do binární haldy, doběhne v čase $\mathcal{O}((n + m) \cdot \log n)$.

Důkaz. Inicializace trvá zase $\mathcal{O}(n)$. Při operacích s binární haldou víme, jak dlouho potrvají (viz tabulka ??). Akorát si nyní musíme rozmyslet, kdy se která operace provádí.

- Při *otevírání* vrcholu provádíme v haldě operaci INSERT⁹. Celkově vložíme do haldy max. všech n vrcholů, což potrvá $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při *uzavírání* vrcholu provádíme operaci EXTRACTMIN (opět max. n -krát), tj. $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při *aktualizaci vzdálenosti* $D(v)$ provádíme DECREASE (tato hodnota se nikdy nemůže zvýšit). To provádíme vždy při kontrole sousedů (tzn. daných hran), kterých je pro všechny vrcholy dohromady max. $\mathcal{O}(m)$ (stejný argument jako při započítávání hran). Tzn. $m \cdot \mathcal{O}(\log n) = \mathcal{O}(m \log n)$.

Po sečtení dostaneme

$$\begin{aligned} \mathcal{O}(n + n \log n + n \log n + m \log n) &= \mathcal{O}(n + 2n \log n + m \log n) \\ &= \mathcal{O}(n + n \log n + m \log n) \\ &= \mathcal{O}(n \log n + m \log n) \\ &= \mathcal{O}((n + m) \cdot \log n). \end{aligned}$$

□

Ačkoliv to není úplně zjevné, výraz $(n + m) \cdot \log n$ roste o dost pomaleji oproti $n^2 + m$, tedy binární haldou si skutečně pomůžeme oproti seznamu.

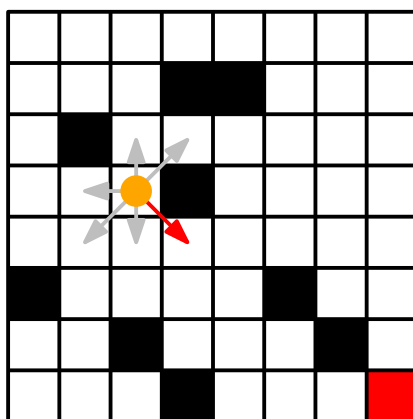
⁹Náš odhad $\mathcal{O}(n \log n)$ by šel dokonce zlepšit i na $\mathcal{O}(n)$, když do něj započítáme fakt, že hladin v haldě je ze začátku 0 a postupně přibývají (tzn. není jich z počátku „moc“), ale celkový odhad složitosti by vyšel stejně a navíc odvození tohoto faktu je již trochu náročnější.

Závěrem zde je ještě uvedme, že často nehledáme cestu do všech vrcholů, ale pouze mezi nějakou konkrétní dvojicí. V takovou chvíli můžeme výpočet algoritmu zarazit ve chvíli, kdy narazíme na cílový vrchol, protože jak již víme, při otevírání vrcholu je $D(v)$ skutečná délka nejkratší cesty. V tomto případě je možné ještě dále urychlit hledání nejkratší cesty, neboť Dijkstrův algoritmus může pak provádět hodně nadbytečné práce. Na to se podíváme v další sekci ??.

1.7 Algoritmus A*

Nyní omezíme naše soustředění na případy, kdy hledáme cestu pouze do nějakého konkrétního vrcholu¹⁰ $v_G \in V$ (nikoliv do všech, jako tomu bylo doposud). Stále pracujeme s grafy, jejichž hrany mají nezáporné ohodnocení. Všechny dosavadně vysvětlené algoritmy by jistě fungovaly i v tomto případě (při otevírání cílového vrcholu jednoduše vyskočíme z hlavního cyklu). Nabízí se však otázka, zdali nemůžeme znalosti cílového vrcholu nějak využít. Pokud bychom např. hledali nejkratší cestu v mřížce s rozmístěnými překážkami (tj. na některá políčka nelze vstoupit), nejpíše by dávalo smysl upřednostňovat políčka, která jsou blíže cílovému políčku, než ty, která jsou od něj dál.

Mějme bludiště, v němž se nachází hráč (oranžový kruh), jež se může pohybovat libovolným směrem vždy o jedno pole, přičemž cílovým pole se nachází vpravo dole (červeně vyznačené). Situaci lze vidět na obrázku ??. Takovou situaci můžeme znázornit pomocí grafu, kde každá hrana má váhu 1, popř. lze použít neohodnocený graf. Pokud bychom aplikovali např. BFS, či Dijkstrův algoritmus, prozkoumali bychom



Obrázek 1.18: Mřížka a optimální směr pohybu hráče.

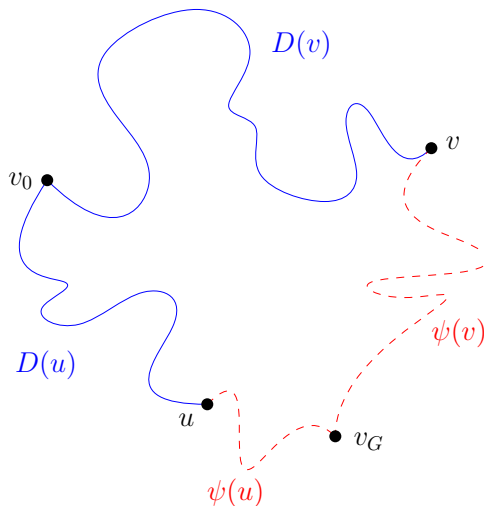
postupně všechny směry (šípky) stejně. Je však zřejmé, že nejvýhodnější je vydat směrem vyznačeným červeně, neboť je nejbližší cíli. V roce 1968 tak přišli Peter Hart, Nils Nilsson a Bertam Raphael s úpravou původního Dijkstrova algoritmu, která nese název A*.

Myšlenka je stále stejná, avšak nyní budeme vybírat vrcholy podle hodnoty $D(v) + \psi(v)$ (místo pouhého $D(v)$), kde ψ je tzv. *heuristická funkce* nebo zkráceně *heuristika*, která slouží jako *odhad vzdálenosti od cílového vrcholu* v grafu (viz obrázek ??). Součtu $D(v) + \psi(v)$ budeme říkat tzv. *f-skóre*¹¹, značíme $f(v)$. V každé iteraci tak budeme vybírat vrchol s nejnižším *f-skóre*, k čemuž lze opět použít pole, nebo binární haldu.

Oproti Dijkstrově algoritmu tedy skutečně nenastává moc změn, akorát je potřeba si zvlášť uchovávat hodnoty *f-skóre*.

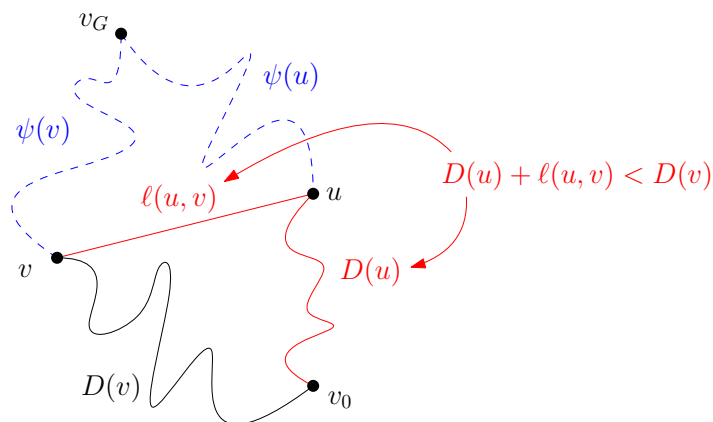
¹⁰ „G“ od angl. *goal*

¹¹ V jiných textech, převážně anglických, se někdy používá pro heuristiku označení h a pro vzdálenost označení g , tzv. *g-skóre*, tj. $f(v) = g(v) + h(v)$.

Obrázek 1.19: Znáznornění heuristické funkce ψ .

1.7.1 Správnost algoritmu A*

Podobně jako u Dijkstrova algoritmu i zde platí, že při výběru (tj. otevírání) vrcholu v odpovídá $D(v)$ délce nejkratší cesty. Argument pro tuto skutečnost je podobný. Předpokládejme, že otevíráme nějaký vrchol v , přičemž hodnota $D(v)$ je větší, než je skutečná délka nejkratší cesty. Pak existuje nějaký sousední vrchol u vrcholu v , přes nějž nutně vede nejkratší cesta do v , tj. platí $D(u) + \ell(u, v) < D(v)$ (viz obrázek ??). Jak to bude s f -skóre vrcholu v ? Těžko říci. Co vůbec víme o funkci ψ ? Zatím jsme si nespécifikovali

Obrázek 1.20: Situace při výběru vrcholu v při heuristice ψ , kdy $D(v)$ neodpovídá délce nejkratší cesty.

žádné vlastnosti, které bychom od ní očekávali. Ukazuje se však, že ψ si libovolně zvolit nemůžeme. Naše heuristika musí být v jistém smyslu „rozumná“. Pokud se vydáme z vrcholu u po hraně do vrcholu v a z něj poté do cílového vrcholu v_G , určitě výsledná cesta bude alespoň tak dlouhá, jako nejkratší cesta z u do v_G . To samé by mělo platit i pro náš odhad vzdálenosti¹², tedy $\psi(u) \leq \ell(u, v) + \psi(v)$.

Pokud tedy budeme předpokládat, že ψ splňuje výše zmíněnou vlastnost, musí pak platit následující:

$$f(u) = D(u) + \underbrace{\psi(u)}_{\leq \ell(u, v) + \psi(v)} \leq \underbrace{D(u) + \ell(u, v)}_{< D(v)} + \psi(v) < D(v) + \psi(v) = f(v).$$

¹²Tomu se obecně říká tzv. *trojúhelníková nerovnost*. Pro libovolnou trojici vrcholů platí $d(u, v) \leq d(u, w) + d(w, v)$, kde d značí vzdálenost daných vrcholů v grafu G .

Algoritmus 1.7.1: A*

Vstup: Nezáporně ohodnocený graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

// Inicializace

```

1 foreach vrchol  $v \in V$  do
2    $stav(v) \leftarrow \text{nenalezený}$ 
3    $D(v) \leftarrow \infty, f(v) \leftarrow \infty$ 
4  $stav(v_0) \leftarrow \text{otevřený}$ 
5  $D(v_0) \leftarrow 0, f(v_0) \leftarrow \psi(v_0)$ 
6 while existují otevřené vrcholy do
7    $v \leftarrow$  otevřený vrchol s nejmenším  $f$ -skóre
8   if  $v = v_G$  then
9     return  $D(v_G)$ 
10  foreach soused  $w$  vrcholu  $v$  do
11    // Našli jsme lepší cestu
12    if  $D(v) + \ell(v, w) < D(w)$  then
13       $D(w) \leftarrow D(v) + \ell(v, w)$ 
14       $f(w) \leftarrow D(v) + \ell(v, w) + \psi(w)$ 
15       $stav(w) \leftarrow \text{otevřený}$ 
16   $stav(v) \leftarrow \text{uzavřený}$ 

```

Výstup: Vzdálenost v_0 od v_G

Z toho je ale již vidět, že $f(u) < f(v)$, tedy u musí mít nižší f -skóre než v . Tzn. algoritmus A* by upřednostnil vrchol u před v .

Upozorníme zde na fakt, že náš předpoklad o heuristice je velmi důležitý, neboť pokud bychom si zvolili heuristiku nevhodně, algoritmus již fungovat nebude a obecně může dojít k tomu, že nějaký z vrcholů vybereme „předčasně“.

Poznámka 1.7.1. • Všimněme si, že Dijkstrův algoritmus je speciálním případem algoritmu A*. Stačí pro všechny vrcholy v položit¹³ $\psi(v) = 0$.

- Je dobré dodat, že heuristiku bychom měli být schopni pro každý vrchol spočítat co nejrychleji, nejlépe v konstantním čase, tj. $\mathcal{O}(1)$.

Rozbor časové složitosti zde vynecháme. Je totiž silně závislá na tom, jakou heuristiku ψ si zvolíme. Při vhodné volbě doběhne A* zpravidla rychleji oproti Dijkstrově algoritmu. Může se však také stát, že ψ zvolíme nevhodně (i přesto, že splňuje trojúhelníkovou nerovnost výše), a algoritmus nakonec poběží déle.

1.7.2 Ukázky heuristických funkcí

Zbývá zodpovědět otázku, jakou heuristiku tedy použít? Odpověď není zcela jednoznačná, neboť záleží na situaci. Vraťme se opět k příkladu s bludištěm, kdy začínáme na určitém políčku a snažíme se dostat do cílového, přičemž hráč se smí pohybovat všemi směry vždy o jedno pole. Políčka identifikujeme podle jejich souřadnic, tj. dvojice (x, y) .

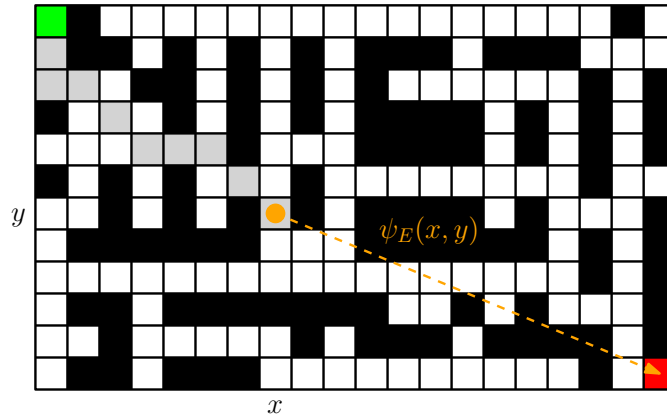
Jednou možností je tzv. *eukleidovská vzdálenost*. Ta je pro libovolnou dvojici bodů v rovině $X = (x_1, y_1)$

¹³Lze zvolit klidně i obecnou konstantu $c \in \mathbb{R}$, tj. $\psi(v) = c$.

a $Y = (x_2, y_2)$ definována jako

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

V případě cílového políčka známe jeho souřadnice, takže můžeme snadno vypočítat eukleidovskou vzdálenost libovolného políčka od cílového. To lze použít jako heuristiku¹⁴, označme si ji ψ_E . Situaci lze sledovat na obrázku ??, kde šedě je vyznačena nejkratší cesta do pole se souřadnicemi (x, y) . Pokud

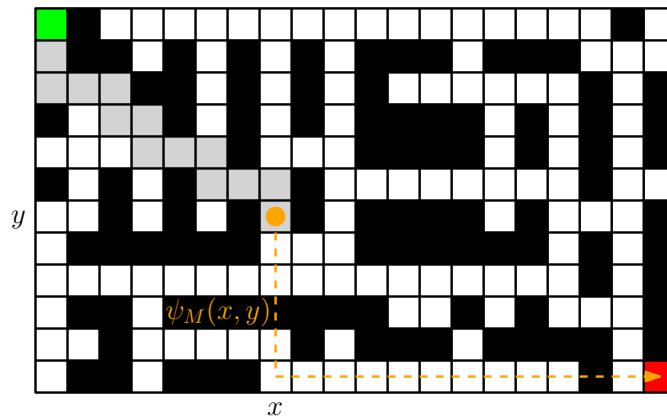


Obrázek 1.21: Příklad bludiště s eukleidovskou heuristikou.

bychom však pracovali s bludištěm, kde se *nelze pohybovat po diagonálách* nebo by nám to pravidla neumožňovala, můžeme výpočet heuristické funkce nahradit něčím jednodušším. V tomto ohledu se pak často využívá tzv. *manhattanská vzdálenost* (označme ψ_M) definovaná vztahem

$$|x_1 - x_2| + |y_1 - y_2|.$$

Stejně jako eukleidovská vzdálenost, i manhattanská vzdálenost splňuje trojúhelníkovou nerovnost.



Obrázek 1.22: Příklad bludiště s manhattnskou heuristikou.

Její výhodou je nižší náročnost při výpočtu (výpočet odmocniny¹⁵ je již trochu pomalejší) a to hlavně z důvodu, že pracujeme vždy s celými čísly. Její nevýhodou však může být, že oproti eukleidovské vzdálenosti nabývá větších hodnot.

¹⁴Dokonce bychom mohli použít jako heuristiku pouze výraz pod odmocninou, tj. $(x_1 - x_2)^2 + (y_1 - y_2)^2$.

¹⁵V mnoha případech, např. v některých starších videohrách jako Quake III, se vývojáři snažili vyhýbat výpočtu odmocniny, nebo volili některé sofistikovanější výpočty, které byly rychlé a poskytovaly uspokojivý odhad hledané hodnoty.

Kapitola 2

Algoritmicky těžké problémy

Mnoho problémů lze v informatice řešit pomocí polynomiálních algoritmů, tzn. běžících v čase $\mathcal{O}(n^k)$ pro nějaké prvné k . Např. problém řazení prvků v poli jsme schopni řešit pomocí různých algoritmů, např. *BubbleSortem*, jehož časová složitost je v nejhorším případě $\mathcal{O}(n^2)$, kde n je počet prvků pole. Podobně hledání nejkratší cesty v grafu dokážeme řešit polynomiálně např. Dijkstrovým algoritmem, který i v případě implementace pomocí pole běží v čase $\mathcal{O}(n^2 + m)$, kde n je počet vrcholů a m počet hran¹. S jistou rezervou lze říci, že polynomiální algoritmy jsou prakticky dobře použitelné.

Bohužel ne vždy je situace takto příznivá. Lze totiž narazit na problémy, na které není známý polynomiální algoritmus a o některých dokonce s jistotou víme, že je v polynomiálním čase řešit nelze. Dobrým příkladem jsou v tomto ohledu např. *Hanojské věže*², kde nejlepší algoritmus je exponenciální, tj. $\mathcal{O}(2^n)$, neboť je vždy potřeba exponenciálně mnoho kroků k vyřešení.

Nás budou zajímat ty nejdůležitější problémy z této oblasti, protože mezi nimi lze nalézt zajímavé vztahy, a posléze si uděláme menší ochutnávku z problematiky P a NP.

2.1 Problém splnitelnosti

Jako první začneme zdánlivě možná jednoduchým problémem, který však na poli informatiky hraje dosti důležitou roli. Předpokládejme, že máme zadanou nějakou logickou formuli φ o libovolném počtu logických proměnných, označme např. $x_1, x_2, \dots, x_n$. Naší otázkou je, jestli je možné hodnoty proměnných $x_1, x_2, \dots, x_n$ nastavit tak (tj. dosadit hodnoty 0 a 1), aby byla formule splněna³, tj. $\varphi(\dots) = 1$. Takovému ohodnocení budeme říkat *splňující*.

SPLNITELNOST OBECNÉ LOGICKÉ FORMULE

Vstup problému: Logická formule φ .

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

¹Může se zdát matoucí, že zde figurují dvě proměnné n a m místo jedné, ale počet hran je omezený (nejvýše mohou být každé dva vrcholy spojeny hranou) a to výrazem $n(n+1)/2$, což je polynom. Tedy všechny prezentované grafové algoritmy běží v polynomiálním čase

²Pro zájemce podrobnější vysvětlení: https://en.wikipedia.org/wiki/Tower_of_Hanoi

³Může se na první pohled zdát, že se jedná o dosti teoretickou záležitost, nicméně později uvidíme, že mnoho jiných problémů má se SAT silnou spojitost.

Nejdříve si zopakujeme logické operace a spojky. Mezi ně řadíme $\neg, \wedge, \vee, \implies, \iff$.

- **Negace** $\neg x$. *Převrací logickou hodnotu x .*
- **Konjunkce** $a \wedge b$. *Pravdivá, jsou-li obě hodnoty operandů a, b pravdivé.*
- **Disjunkce** $a \vee b$. *Pravdivá, je-li alespoň jedna z hodnot operandů a, b pravdivá.*
- **Implikace** $a \implies b$. *Je-li předpoklad a splněn, je splněn i závěr b . Tj. implikace je nepravdivá, pokud platí a a neplatí b .*
- **Ekvivalence** $a \iff b$. *a platí právě tehdy, když platí b . Tj. ekvivalence je pravdivá, jsou-li hodnoty operandů a, b současně 0 nebo 1.*

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$a \implies b$	$a \iff b$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Zde konstatujeme, že ve všech příkladech bude mít negace $\neg$ vždy *nejvyšší prioritu* oproti ostatním operacím. Prioritu ostatních operací budeme stanovovat pomocí závorek.

Příklad 2.1.1. • $\varphi(x) = x \wedge \neg x \dots$ **Není splnitelná**, pro $x = 0$ i $x = 1$ je $\varphi = 0$.

- $\psi(x) = x \vee \neg x \dots$ **Je splnitelná**, a to jak pro $x = 0$, tak $x = 1$.
- $\chi(a, b, c) = ((a \wedge b) \vee \neg c) \implies (\neg a \wedge \neg c) \dots$ **Je splnitelná**, např. pro $a = 0, b = 0$ a $c = 0$.

V příkladu ?? výše se jedná o poměrně jednoduché instance, kdy jsme schopni vidět hned, zda je formule splnitelná. Pokud bychom však uvažili nějakou složitější formuli, problém se již značně zkomplikuje.



Obecně pro formuli o n proměnných je třeba prozkoumat řádově 2^n možných ohodnocení. Naivní algoritmus zkoušející všechny možnosti je tak *exponenciální*.

Pro SAT není dodnes známý žádný algoritmus, který by jej uměl řešit pro libovolnou logickou formuli v polynomiálním čase.

2.1.1 Konjunktivní normální forma

Zkusíme se podívat na formule ve speciálním tvaru, a to tzv. *konjunktivní normální formě*, neboli zkráceně *CNF*.⁴

Každá formule skládá z tzv. **klauzulí** obsahujících tzv. **literály**, což je buď *proměnná*, nebo *její negace*.

- Klauzule jsou ve formuli odděleny *logickou spojkou* $\wedge$
- a v každé klauzuli jsou literály odděleny *logickou spojkou* $\vee$.

$$\varphi(\dots) = \underbrace{(\dots \vee \dots \vee \dots)}_{\text{klauzule}} \wedge (\dots \vee \overbrace{x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \overbrace{\neg x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \underbrace{y}_{\text{literál}} \vee \dots) \wedge \dots$$

⁴Z angl. *conjunctive normal form*.

Příklad 2.1.2. • $\varphi(x, y, z) = \neg x \wedge (y \vee z) \dots$ **Je v CNF.** Formule φ obsahuje klauzule x (klauzule o jednom literálu) a $y \vee z$.

- $\psi(a, b) = a \wedge b \dots$ **Je v CNF.**
- $\chi(a, b, c, d) = a \wedge (b \vee (c \wedge d)) \dots$ **Není v CNF**, protože výraz $c \wedge d$ není literál.

Poslední formuli lze však převést do CNF: $a \wedge (b \vee c) \wedge (b \vee d)$.

Z příkladu ?? se tak nabízí otázka, zda je možné *libovolnou formuli* převést do CNF. Existuje vůbec pro každou formuli taková ekvivalentní⁵ formule? Ale ano, v logice se i dokazuje, že každé formuli existuje ekvivalentní formule v CNF.

Věta 2.1.3. Pro každou logickou formuli ψ existuje ekvivalentní formule ψ' v CNF.

Toto tvrzení lze, pochopitelně, dokázat, ale zde se tím zabývat nebudeme a zkrátka jej přijmeme jako fakt. Podstatná věc, která z toho plyne, je, že pokud je nějaká formule ψ splnitelná, pak je splnitelná i jí ekvivalentní formule ψ' v CNF a naopak pokud ψ není splnitelná, není splnitelná ani ψ' . Symbolicky

$$\psi \text{ je splnitelná} \iff \psi' \text{ je splnitelná.}$$

2.1.2 Převod do CNF pomocí tabulky

Podívejme se na způsob, jak lze převést libovolnou logickou formuli do CNF.

Máme obecnou formuli φ s proměnnými $x_1, x_2, \dots, x_n$, pro kterou budeme konstruovat ekvivalentní formuli φ' v CNF. Vycházíme z tabulky pravdivostních hodnot, přičemž pro každé **nepravdivé** ohodnocení vytvoříme *klauzuli* s n literály, kde obecně i -tý literál je

- x_i , pokud v daném ohodnocení je $x_i = 0$,
- jinak $\neg x_i$ (tj. když $x_i = 1$).

Podívejme se na úvod na příklad ?? níže.

Příklad 2.1.4. Máme formuli $\psi(x, y, z) = (x \implies y) \implies z$, pro kterou chceme najít ekvivalentní formuli ψ' v CNF. Nejprve si tedy sestavíme tabulku pravdivostních hodnot. Z tabulky můžeme vidět,

x	y	z	$x \implies y$	$(x \implies y) \implies z$
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

že formule φ je nepravdivá pro

- $x = 0, y = 0, z = 0$,
- $x = 0, y = 1, z = 0$

⁵Libovolné formule φ a ψ jsou ekvivalentní, pokud pro stejné ohodnocení proměnných platí $\varphi(\dots) = \psi(\dots)$.

- a $x = 1, y = 1, z = 0$.

Tedy celkově sestavíme 3 klauzule:

- $(\overbrace{x}^{x=0} \vee \underbrace{y}_{y=0} \vee \overbrace{z}^{z=0})$,
- $(x \vee \underbrace{\neg y}_{y=1} \vee z)$,
- $(\neg x \vee \neg y \vee z)$.

Výsledná formule v CNF bude mít tvar:

$$\psi'(x, y, z) = (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee z).$$

Čtenář se sám může přesvědčit, že formule ψ a ψ' pro libovolné ohodnocení mají stejnou pravdivostní hodnotu.

Nabízí se otázka: „proč tento postup funguje?“ Pokud nějaká formule v CNF není splněna pro určité ohodnocení, pak to nutně znamená, že *alespoň jedna z klauzulí není splněna*. Každá klauzule však obsahuje literály spojené logickou spojkou $\vee$ a tedy existuje pro ni jen jediné ohodnocení daných literálů, tak, aby nebyla splněna. Z tohoto principu vychází ona konstrukce. Vybereme si ta ohodnocení původní formule, která ji nesplňují a sestrojíme takovou klauzuli, aby právě pro dané ohodnocení nebyla splněna.

Příklad 2.1.5. $\chi(x_1, x_2, x_3, x_4) = (x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$.

x_1	x_2	x_3	x_4	$x_1 \iff x_2$	$\neg(x_3 \iff x_4)$	$(x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$
0	0	0	0	1	0	1
0	0	0	1	1	1	1
0	0	1	0	1	1	1
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	1	1
0	1	1	0	0	1	1
0	1	1	1	0	0	0
1	0	0	0	0	0	0
1	0	0	1	0	1	1
1	0	1	0	0	1	1
1	0	1	1	0	0	0
1	1	0	0	1	0	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	0	1

$$\chi'(x_1, x_2, x_3, x_4) = (x_1 \vee x_2 \vee \neg x_3 \vee x_4) \wedge (x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3 \vee \neg x_4).$$



Problém: Generování tabulky (resp. procházení všech možných ohodnocení) nás přivádí zpět k původnímu problému, a to sice *exponenciální časové složitosti*.

V tomto případě však nejsme schopni postup učinit více efektivní, neboť ve skutečnosti se může stát, že pro obecnou logickou formuli se může jí ekvivalentní formule v CNF až exponenciálně prodloužit.⁶

2.1.3 Tseitinova transformace

Značně rychlejší způsob převodu formule do CNF předvedl ruský informatik *Grigori Tseitin*. Problém s předchozím postupem v podsekcí ?? je ten, že se v nové formuli ψ snažíme ponechat *pouze* proměnné v původní formuli φ , což způsobí, že formule se může až exponenciálně prodloužit. Postup, který si nyní ukážeme, zavádí do výsledné formule některé proměnné navíc, avšak za cenu rychlejšího algoritmu.

Postup Tseitinovy transformace formule φ lze rozdělit do celkem 4 kroků.

1. Rozdělíme formuli φ na jednotlivé podformule.
2. Pro každou dílčí podformuli φ_i si zavedeme novou proměnnou y_i . Speciálně y_1 je logicky ekvivalentní s celou původní formuli φ , tj. $y_1 \iff \varphi$.
3. Vytvoříme novou formuli ψ následovně:

$$\psi = y_1 \wedge (y_1 \iff \varphi_1) \wedge (y_2 \iff \varphi_2) \wedge \cdots \wedge (y_n \iff \varphi_n).$$

Hned se nabízí námitka, že nová formule ψ není v CNF, avšak stačí si uvědomit, že logickou ekvivalenci lze přepsat pomocí $\wedge$ a $\vee$ následovně:

$$a \iff b = (a \implies b) \wedge (b \implies a) = (\neg a \vee b) \wedge (\neg b \vee a). \quad (2.1)$$

Nové proměnné $y_1, y_2, \dots, y_n$ v podstatě indikují, jaké hodnoty nabývá daný logický výraz v původní formuli pro nějaké konkrétní ohodnocení původních proměnných. Chceme-li tedy zjistit, jaké logické hodnoty nabývá daná formule φ pro určité logické hodnoty jejích proměnných, stačí novým proměnným přiřadit logickou hodnotu podle toho, jaké hodnoty nabývá výraz φ_i , který reprezentují. Přesně proto jsme ostatně konstruovali podformule $y_i \iff \varphi_i$.

Pojďme se podívat na konkrétní příklad.

Příklad 2.1.6. Máme formuli $\alpha(p, q, r) = ((p \vee q) \wedge r) \implies \neg p$. Chceme pro ni sestavit Tseitinovou transformaci novou formuli β .

Formule α obsahuje tyto dílčí podformule:

- $\neg p$,
- $p \vee q$,
- $(p \vee q) \wedge r$,
- a $((p \vee q) \wedge r) \implies \neg p$ (původní formule α).

Pro ně si zřídíme nové proměnné y_1, y_2, y_3 a y_4 . Tzn. do nové formule β přidáme výrazy

- $y_1 \iff ((p \vee q) \wedge r) \implies \neg p$,
- $y_2 \iff (p \vee q) \wedge r$,
- $y_3 \iff p \vee q$
- a $y_4 \iff \neg p$.

⁶Stačí např. vzít formuli, která je *nesplnitelná*, tj. pro všechna možná ohodnocení platí $\psi(\dots) = 0$. Celkově by tak ekvivalentní formule měla až 2^n klauzulí.

Výsledná formule bude tedy vypadat následovně:

$$\begin{aligned}\beta(p, q, r, y_1, y_2, y_3, y_4) &= y_1 \wedge (y_1 \iff ((p \vee q) \wedge r) \implies \neg p) \\ &\quad \wedge (y_2 \iff ((p \vee q) \wedge r)) \\ &\quad \wedge (y_3 \iff (p \vee q)) \\ &\quad \wedge (y_4 \iff \neg p).\end{aligned}$$

Ekvivalence ve formuli bychom mohli přepsat pomocí vztahu (??), který jsme si ukazovali výše.

Zkusme nyní formuli vyhodnotit např. pro ohodnocení proměnných $p = 1, q = 0$ a $r = 0$. Původní formule je rovna:

$$\alpha(1, 0, 0) = ((1 \vee 0) \wedge 0) \implies \neg 1 = (1 \wedge 0) \implies \neg 1 = 0 \implies 1 = 1.$$

Hodnoty nových proměnných nastavíme tedy následovně:

- $y_4 = 0$, protože $\neg p = 0$,
- $y_3 = 1$, protože $p \vee q = 1$,
- $y_2 = 0$, protože $(p \vee q) \wedge r = 0$,
- a $y_1 = 1$, protože $((p \vee q) \wedge r) \implies \neg p = 1$.

Nyní můžeme vyhodnotit novou formuli β :

$$\begin{aligned}\beta(1, 0, 0, 1, 0, 1, 0) &= 1 \wedge (1 \iff ((1 \vee 0) \wedge 0) \implies \neg 1) \wedge (0 \iff ((1 \vee 0) \wedge 0)) \\ &\quad \wedge (1 \iff (1 \vee 0)) \wedge (0 \iff \neg 1) \\ &= 1 \wedge 1 \wedge 1 \wedge 1 \wedge 1 = 1.\end{aligned}$$

Lze se tedy přesvědčit, že formule β má stejnou logickou hodnotu jako původní formule α .

Obecně vždy bude platit, že nově vzniklá formule je splnitelná, právě tehdy když je splnitelná i původní formule. Ovšem podstatnou výhodou oproti postupu s tabulkou je ta, že nová formule je podstatně kratší a lze ji tedy sestavit mnohem rychleji.



Pro každý z podvýrazů φ_i sestavíme klauzuli $y_i \iff \varphi_i$, tzn. celkově jich bude lineárně mnoho v počtu podvýrazů.

Při hlubší analýze tohoto postupu (resp. algoritmu) se lze přesvědčit, že velikost výsledné formule ψ je **lineární** ve velikosti původní formule φ .

2.2 Problém SAT

Již jsme v podsekcí ?? nahlédli, že převod do obecné formule do CNF “hrubou silou” je problematický, neboť se nová formule může až exponenciálně prodloužit. V podsekcí ?? jsme si proto ukázali Tseitinovu transformaci, která původní postup zefektivnila přidáním nových proměnných do výsledné formule. To znamená, že každou formuli jsme schopni efektivně převést do CNF. Zkusme si tedy původní problém zjednodušit. Předpokládejme nyní, že formule, které máme na vstupu, jsou již v CNF. Tento problém je v informatice velmi význačný a označuje se zkratkou SAT (z angl. *satisfiability*, neboli splnitelnost).

SPLNITELNOST LOGICKÉ FORMULE V CNF (SAT)

Vstup problému: Logická formule φ v CNF.

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nyní se již skutečně zaměříme na samotnou splnitelnost dané formule na vstupu. Pro tento účel se obecně využívají tzv. *SAT solvery*, tedy algoritmy, které na vstupu dostanou *formuli* φ v CNF, o níž rozhodnou, zda je či není splnitelná. Přitom využívají formy, v níž je formule zapsaná. Pro tento účel si ukážeme tzn. *algoritmus DPLL* pojmenovaný po svých autorech⁷, kteří s ním přišli roku 1961.

2.2.1 Algoritmus DPLL

Tento algoritmus je založený na rekurzivním postupu. Na vstupu algoritmu je libovolná formule φ v CNF, kterou algoritmus postupně vyhodnocuje. Využívá dvojici procedur – **Unit propagation** a **Pure literal elimination**.

Unit propagation

Tato procedura přijímá hledá v dané formuli φ tzn. *jednotkovou klauzuli*. To je klauzule, která obsahuje **pouze jeden literál**. Např. formule ω níže obsahuje jednotkovou klauzuli.

$$\omega(x, y) = (x \vee \neg y) \wedge \underbrace{\neg y}_{\text{jednotková kl.}} \wedge (\neg x \vee y).$$

Pokud se taková klauzule nachází ve formuli, lze toho využít, neboť pro danou proměnnou existuje vždy jen jediné ohodnocení, takové, aby celá formule byla splněna (každá klauzule musí být splněna). Obecně pokud formule φ obsahuje jednotkovou klauzuli s proměnnou x , pak mohou nastat dva případy. Označme *varphi'* zbytek klauzulí ve formuli φ .

1. Pokud má formule tvar $\varphi = x \wedge \varphi'$, pak nutně $x = 1$.
2. Jinak pokud má formule tvar $\varphi = \neg x \wedge \varphi'$, pak musí být $x = 0$.

Fakt, že jsme schopni hned rozhodnout, jakou ohodnocení musí mít daná proměnná, je v tomto případě hodně ku prospěchu, neboť formuli můžeme zjednodušit. Protože ohodnocení dané jednotkové klauzule již známe, můžeme ji z formule φ odstranit. Zároveň můžeme **odstranit všechny klauzule**, které se vyhodnotí díky proměnné x na 1, neboť nemohou již nijak ovlivnit logickou hodnotu zbylých klauzulí.

Např. pokud formule obsahuje literál x v jednotkové klauzuli, pak hodnota proměnné musí být 1. Tzn. klauzule obsahující x se vyhodnotí též na 1. tj.

$$(\dots \vee x \vee \dots) = 1.$$

Naopak pokud obsahuje literál $\neg x$, pak nutně $x = 0$ a tedy

$$(\dots \vee \neg x \vee \dots) = 1.$$

Dále ještě odstraníme výskyty literálu x , resp. $\neg x$ v *opačné polaritě*, tedy negaci původního literálu. Tzn. pokud literál v jednotkové klauzuli byl x , pak opačná polarita literálu je $\neg x$ a naopak. Důvod, proč jej můžeme odstranit, je ten, že v rámci kterékoliv jiné klauzule se daný literál v opačné polaritě vyhodnotí

⁷Martin (D)avid, Davis (P)utnam, George (L)ogemann a Donald W. (L)oveland.

vždy na 0. Protože je však spojen s ostatními literály pomocí $\vee$, nemůže nijak ovlivnit výslednou logickou hodnotu klauzule a tudíž jej můžeme odstranit. Tedy např. pokud $x = 1$, pak můžeme klauzule tvaru

$$(\cdots \vee y \vee \underbrace{\neg x}_{\text{odstraníme}} \vee z \vee \cdots)$$

zjednodušit na

$$(\cdots \vee y \vee z \vee \cdots).$$

Podobně pro $x = 0$:

$$(\cdots \vee y \vee x \vee z \vee \cdots) = (\cdots \vee y \vee z \vee \cdots).$$

Příklad 2.2.1. Máme formuli

$$\chi(x, y, z) = (x \vee y) \wedge y \wedge (z \vee \neg y) \wedge (\neg x \wedge \neg y \vee \neg z).$$

Aplikujme na ni proceduru *Unit propagation*. Můžeme si všimnout, že klauzule y je jednotková a proměnná y tedy musí být nastavena na 1.

Protože $y = 1$, je zároveň splněna i klauzule $x \vee y$, tj. můžeme ji odstranit. Zároveň můžeme odstranit literál $\neg y$ ze zbylých klauzulí $z \vee \neg y$ a $\neg x \wedge \neg y \vee \neg z$. Celkově dostaneme novou zjednodušenou formuli

$$\chi'(x, y, z) = z \wedge (\neg x \vee \neg z).$$

Pure literal elimination

Ve vstupní formuli φ může nastat situace, kdy se zde určitá proměnná nachází pouze v jedné polaritě. Tzn. pokud se ve formuli nachází proměnná x , pak se v ní nikde nenachází $\neg x$ a naopak pokud se v ní nachází $\neg x$, pak v ní nikde není x . V obou případech můžeme rovnou určit hodnotu dané proměnné, neboť tím splníme všechny klauzule, v nichž se vyskytuje. Ty můžeme odstranit, neboť jsou tím pádem splněny.

$$\cdots \wedge \overbrace{(\cdots \vee \underbrace{x}_1 \vee \cdots)}^1 \wedge \cdots \wedge \overbrace{(\cdots \vee \underbrace{x}_1 \vee \cdots)}^1 \wedge \cdots$$

Podobně pro negaci, tj.

$$\cdots \wedge \overbrace{(\cdots \vee \neg \underbrace{x}_0 \vee \cdots)}^1 \wedge \cdots \wedge \overbrace{(\cdots \vee \neg \underbrace{x}_0 \vee \cdots)}^1 \wedge \cdots$$

Poté, co jsme si vysvětlili základní dvojici procedur, se nyní nabízí trojice případů, které je ještě třeba prozkoumat.

- *Co když postupným odstraňováním proměnných mi ve formuli vznikne prázdná klauzule?* Taková situace může nastat v případě procedury Unit propagation, kdy odstraňujeme proměnné v opačné polaritě. Pokud nastane situace, že je nějaká klauzule prázdná, pak to znamená, že daná klauzule pod zvoleným ohodnocením **není splnitelná** a tudíž ani celá formule. V takovou chvíli můžeme prohlásit formuli za **nesplnitelnou pod daným ohodnocením**⁸.
- *Co když odstraníme postupně všechny klauzule?* K takové situaci může dojít, pokud jsou všechny klauzule splněny. V takovou určitě chvíli víme, že je původní formule **splnitelná**.

⁸Je potřeba si uvědomit, že ohodnocení proměnných si během výpočtu volíme, tedy v takovém případě to ještě nutně neznamená, že je formule celkově nesplnitelná. Pouze víme, že dané ohodnocení určitě není splňující

- Co když po aplikaci obou procedur stále nevíme, zda je φ splnitelná? V takovou chvíli nám nezbývá nic jiného, než si náhodně zvolit nějaký literál a prozkoumat obě možnosti jeho ohodnocení. Náhodně si tedy vybereme literál ℓ a zkusíme jej nastavit jednou na hodnotu 1 a podruhé na hodnotu 0. Tím můžeme formuli φ zjednodušit a nově vzniklou formuli φ' rekurzivně aplikovat stejný algoritmus. Pokud lze splnit zbytek formule φ' , pak lze splnit i původní formuli φ .

Toho můžeme dosáhnout tak, že za formuli φ přidáme uměle jednotkovou klauzuli ℓ , resp. v druhém případě $\neg\ell$. Tuto jednotkovou klauzuli si pak opětovně vybere procedura Unit propagation, která vyhodnotí ℓ buď na 1, nebo na 0. Tzn. zavoláme algoritmus DPLL rekurzivně na formule

$$\varphi \wedge \ell \quad \text{a} \quad \varphi \wedge \neg\ell.$$

S těmito informacemi můžeme dát dohromady pseudokód algoritmu.

Algoritmus 2.2.1: DPLL

Vstup: Formule φ v CNF

```

1 while existuje jednotková klauzule ( $\ell$ ) ve formuli  $\varphi$  do
2    $\varphi \leftarrow \text{UnitPropagation}(\ell, \varphi)$ 
3 while existuje literál  $\ell$  v jediné polaritě ve  $\varphi$  do
4    $\varphi \leftarrow \text{PureLiteralElimination}(\ell, \varphi)$ 
   // Všechny klauzule jsou pravdivé
5 if je  $\varphi$  prázdná then return 1;
   // Všechny literály v klauzuli jsou nepravdivé
6 if obsahuje  $\varphi$  prázdnou klauzuli then return 0;
   // Zkusíme ohodnocení pro  $\ell = 0$  a  $\ell = 1$ 
7 Vyber náhodně literál  $\ell$ 
8 return DPLL( $\varphi \wedge \ell$ )  $\vee$  DPLL( $\varphi \wedge \neg\ell$ )
Výstup: 1, je-li formule  $\varphi$  splnitelná, jinak 0.
```

Zkusme si algoritmus odsimulovat na příkladu.

Příklad 2.2.2. Máme formuli

$$\gamma(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge \neg x_4 \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

Chceme pomocí algoritmu DPLL rozhodnout, zda je splnitelná.

Když se podíváme na pseudokód ?? výše, tak první máme aplikovat proceduru *Unit propagation*. Hledáme tedy jednotkové klauzule ve formuli γ , dokud nějaké existují.

- První jednotková klauzule, které si můžeme všimnout, je $\neg x_4$. Hodnota proměnné x_4 tedy nutně musí být 0. Klauzuli tedy odstraníme. Proměnná se nachází ve formuli pouze jednou, tedy daný literál v opačné polaritě nikde odstraňovat nemusíme. Máme nyní formuli

$$\gamma'(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

- Další jednotková klauzule je x_1 . Proměnná x_1 musí být nutně 1 a klauzuli můžeme odstranit. Dále si však můžeme všimnout, že tím pádem splněna i klauzule $(x_1 \vee x_3)$. Tu můžeme též odstranit. Literál x_1 se dále nachází v opačné polaritě v klauzuli $(\neg x_1 \vee \neg x_3 \vee x_3)$. Literál $\neg x_1$ můžeme také odstranit z formule. Tím se nám formule γ' zjednoduší na

$$\gamma''(x_1, x_2, x_3, x_4) = (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_3).$$

Nově vzniklá formule γ'' již žádné další jednotkové klauzule neobsahuje. Dále tedy můžeme aplikovat proceduru *Pure literal elimination*.

- Formule γ'' obsahuje pouze jeden literál v jediné polaritě, a to $\neg x_2$. Tím pádem hodnotu x_2 můžeme nastavit na 0, čímž splníme dvojici klauzulí $(\neg x_2 \vee \neg x_3)$ a $(\neg x_2 \vee x_3)$. Ty můžeme odstranit, čímž dostaneme formuli

$$\gamma'''(x_1, x_2, x_3, x_4) = \neg x_3 \vee x_3.$$

Protože formule γ''' není ani prázdná, ani neobsahuje prázdnou klauzuli, vybereme náhodně jeden literál a prozkoumáme jeho obě možná ohodnocení. V tomto případě nám zbývají pouze literály x_3 a $\neg x_3$. Vyberme např. literál $\neg x_3$. Nyní máme sestavit dvojici nových formulí $(\neg x_3 \vee x_3) \wedge \neg x_3$ a $(\neg x_3 \vee x_3) \wedge x_3$, na které rekurzivně zavoláme DPLL.

- **Formule** $(\neg x_3 \vee x_3) \wedge \neg x_3$. Opět začneme aplikací Unit propagation. K původní formuli γ''' jsme uměle přidali jednotkovou klauzuli $\neg x_3$. Z jí je zjevné, že x_3 musí být 0. Tím zároveň splníme i klauzuli $(\neg x_3 \vee x_3)$, kterou můžeme odstranit, čímž dostáváme prázdnou formuli. Pure literal elimination aplikovat nemůžeme. Protože je nově vzniklá formule prázdná, vrátíme na výstup 1.
- **Formule** $(\neg x_3 \vee x_3) \wedge x_3$. I zde začneme eliminací jednotkových klauzulí. Tu zde představuje literál x_3 , jehož ohodnocení tedy musí být 1. Tím opět splníme i klauzuli $(\neg x_3 \vee x_3)$, jejímž odstraněním dostaneme zase prázdnou formuli. Tedy opět vrátíme 1.

Obě ohodnocení pro x_3 jsou tedy splňující a tedy i původní formule γ je *splnitelná*.

Ukažme si ještě jeden jednodušší příklad.

Příklad 2.2.3. Mějme formuli $\tau(x, y, z) = x \wedge (\neg x \vee y \vee z) \wedge \neg x$. Chceme opět pomocí DPLL určit splnitelnost.

Jako první jednotkovou klauzuli můžeme vzít x . Tedy můžeme ji odstranit společně se všemi výskyty daného literálu v opačné polaritě, tj. $\neg x$. Tzn. dostaneme formuli

$$\tau'(x, y, z) = (y \vee z) \wedge ().$$

Všimněte si, že odstraněním literálu $\neg x$ jsme dostali ve formuli prázdnou klauzuli⁹ $()$. Aplikací Pure literal elimination se zbavíme i literálů y a z .

$$\tau''(x, y, z) = ()$$

Formule τ'' obsahuje pouze prázdnou klauzuli. V tuto chvíli algoritmus prohlásí formuli τ za *nesplnitelnou*.

Poslední otázka, která se nabízí, je (jako obvykle) časová složitost algoritmu. Vezmeme-li v potaz nejhorší případ, časová složitost bohužel nevychází úplně příznivě.

Věta 2.2.4 (Složitost DPLL). Algoritmus DPLL nad formulí φ o n literálech doběhne v čase $\mathcal{O}(2^n)$ a spotřebuje paměť $\mathcal{O}(n)$.

Důkaz. Nyní je třeba se zamyslet, jaká nejhorší situace může při zpracovávání formule φ nastat. Pokud ve formuli neexistuje žádná jednotková klauzule, ani žádný z literálů není pouze v jedné polaritě (tedy procedury Unit propagation a Pure literal elimination nemůžeme vůbec aplikovat), pak musíme náhodně

⁹Z dané jednotkové klauzule jsme odebrali daný literál v rámci procedury Unit propagation, neprovedli jsme smazání klauzule jako takové.

vybrat nějaký z literálů ℓ a rekurzivně zkusit obě možnosti ohodnocení. Pokud toto nastane v každém volání DPLL, vždy eliminujeme pouze jeden literál. Uvažujme, že jedno volání funkce DPLL obecně zabere c kroků, přičemž následně voláme funkci *dvakrát* na formuli o $n - 1$ literálech (jeden jsme eliminovali). Pokud si označíme počet kroků nad formulí o n literálech $T(n)$, pak můžeme dojít k následujícímu vztahu:

$$T(n) = \underbrace{c}_{\text{zpracování } \varphi} + 2 \cdot \overbrace{T(n-1)}^{\text{rekurze}}.$$

Tomuto typu rovnice se v matematice říká tzv. *rekurentní* či *rekurzivní* rovnice¹⁰, neboť hodnota $T(n)$ přímo závisí na předešlé hodnotě $T(n-1)$. Určitě víme, že prázdnou formuli umíme zpracovat okamžitě, tj. $T(0) = \mathcal{O}(1)$.

Lze ukázat¹¹, že platí $T(n) = \mathcal{O}(2^n)$.

Na formuli o n literálech spotřebuje algoritmus lineární paměť. □

S exponenciální časovou složitostí se sotva spokojíme, avšak v tomto případě je dobré si říci, že zkoumáme složitost v nejhorším případě a v praxi bývají zkoumané logické formule poměrně rychle řešitelné pomocí DPLL.



Problém je však dodnes ten, že na rozhodnutí splnitelnosti logické formule (i těch v CNF) není známý žádný algoritmus, který by běžel v polynomiálním čase, tj. $\mathcal{O}(n^k)$, pro nějaké k . K tomuto tématu se ještě vrátíme v sekci ??.

¹⁰Jedná se vlastně o posloupnost zadanou rekurentně.

¹¹*Pro zájemce:* U rekurentní rovnice se snažíme najít předpis pro $T(n)$, takový, aby jeho hodnota nezávisela na jiných hodnotách T . Řešení obsahuje práci s geometrickými řadami. Víme, že platí $T(n) = c + 2 \cdot T(n-1)$. Zkusme podle tohoto předpisu rozepsat $T(n-1)$.

$$T(n) = c + 2 \cdot T(n-1) = c + 2 \cdot (c + 2 \cdot T(n-2)) = 3c + 4 \cdot T(n-2)$$

Zkusme rozepsat opět i $T(n-2)$.

$$T(n) = 3c + 4 \cdot T(n-2) = 3c + 4 \cdot (c + 2 \cdot T(n-3)) = 7c + 8 \cdot T(n-3)$$

Obecně lze pororovat, že k rozepsáních dostaneme pro $T(n)$ následující vzorec:

$$T(n) = c \cdot \sum_{i=0}^{k-1} 2^i + 2^k \cdot T(n-k).$$

Je důležité si nyní uvědomit, že jedinou hodnotu, kterou známe, je $T(0) = \mathcal{O}(1)$. Pokud vzorec rozepíšeme n -krát, tj. $k = n$, můžeme se tak zbavit rekurence v rovnici.

$$T(n) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n \cdot T(0) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n \cdot \mathcal{O}(1) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n$$

Řada $\sum_{i=0}^{n-1} 2^i$ je geometrická s kvocientem $q = 2$ a prvním členem rovnému $2^0 = 1$, přičemž celkový počet členů je n . Můžeme na ni aplikovat známý vzorec pro součet.

$$T(n) = c \cdot \frac{2^n - 1}{2 - 1} + 2^n = c \cdot (2^n - 1) + 2^n = (c + 1)2^n - c.$$

Protože nás však zajímá asymptotické chování algoritmu, konstanty $c + 1$ a c můžeme zanedbat. Z toho již můžeme vidět, že složitost je exponenciální.

$$T(n) = \mathcal{O}((c + 1)2^n - c) = \mathcal{O}(2^n)$$

2.3 Problém 3-SAT

Zkusme nyní původní problém ještě zjednodušit. Přidáme podmínku, že každá klauzule může obsahovat nejvýše 3 literály. Tím dostáváme variantu problému SAT známý jako *3-SAT*. O formuli, která splňuje zmíněnou formu, budeme říkat, že je v 3-CNF.

SPLNITELNOST LOGICKÉ FORMULE V 3-CNF (3-SAT)

Vstup problému: Formule φ v 3-CNF

Výstup problému: 1, je-li φ splnitelná, jinak 0.

Nabízí se otázka, jestli jsme si skutečně nějak pomohli. Zdánlivě ano, neboť máme kratší klauzule, avšak 3-SAT představuje, překvapivě, stejně těžkou úlohu jako původní problém SAT. Proč? Lze totiž ukázat, že pokud bychom uměli rychle řešit 3-SAT, uměli bychom rychle řešit i původní SAT a naopak. To ukážeme tak, že jeden problém v jistém smyslu “převést” na ten druhý. Pro určitou instanci (tedy formuli) problému SAT můžeme sestavit odpovídající instanci problému 3-SAT, jejímž vyřešením získáme odpověď i na původní problém. Podívejme se na tuto situaci blíže v podsekcí

2.3.1 Převod 3-SAT na SAT

Popíšeme si nyní obecný způsob, jak převést libovolnou formuli v CNF (instance problému SAT) na formuli v 3-CNF (instance problému 3-SAT). Předpokládejme, že máme zadanou nějakou formuli φ v CNF a chceme pro ni sestavit formuli ψ v 3-CNF, takovou, aby byla splnitelná právě tehdy, když je splnitelná i původní formule φ . Pro formuli φ mohou nastat 2 případy:

1. formule obsahuje klauzule délky 3 a kratší,
2. nebo ve formuli existuje klauzule délky $k > 3$.

V prvním případě není co řešit, neboť φ již je v 3-CNF, tedy stačí položit $\psi = \varphi$ a jsme hotovi. V druhém případě musíme dlouhou klauzuli o k literálech rozložit na kratší. Tuto klauzuli¹² rozložíme na dvojici nových klauzulí pomocí nové proměnné x . Označme si literály dané dlouhé klauzule $\ell_1, \ell_2, \dots, \ell_k$. První dvojici ℓ_1 a ℓ_2 si označíme α a zbytek β .

$$\underbrace{\ell_1 \vee \ell_2}_{\alpha} \vee \overbrace{\ell_3 \vee \dots \vee \ell_k}^{\beta} = \alpha \vee \beta$$

Nyní přidáme novou proměnnou x a klauzuli $\alpha \vee \beta$ rozdělíme na dvojici klauzulí následovně:

$$(\alpha \vee x) \wedge (\beta \vee \neg x).$$

Podívejme se nyní na délky nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

- Klauzule $\alpha \vee x = \ell_1 \vee \ell_2 \vee x$ obsahuje 3 literály, takže s ní dále nic provádět nemusíme.
- Klauzule $\beta \vee \neg x = \ell_3 \vee \dots \vee \ell_k \vee \neg x$ obsahuje $k - 1$ literálů. To znamená, že jsme zkrátily původní klauzuli o jeden literál. Pokud $k - 1 > 3$, můžeme postup pro tuto klauzuli opakovat.

Nyní zbývá dořešit, jak to bude s hodnotou proměnné x . Podobně jako u Tseitinovy transformace (viz podsekcí ??), i zde bude její hodnota záviset na hodnotách ostatních proměnných, konkrétně výrazu α .

¹²Pokud φ obsahuje více dlouhých klauzulí, postup opakujeme pro každou z nich zvlášť.

- Pokud $\alpha = 1$, pak nastavíme $x = 0$. Původní klauzule $\ell_1 \vee \dots \vee \ell_k = \alpha \vee \beta$ je splněna díky α . Nová dvojice klauzulí je pak splněna pomocí α a x :

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (1 \vee 0) \wedge (\beta \vee 1) = 1 \wedge 1 = 1.$$

- Pokud $\alpha = 0$, pak nastavíme $x = 1$. Klauzule $\alpha \vee \beta$ musí být splněna díky β . Potom platí:

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (1 \vee 0) = 1 \wedge 1 = 1.$$

Pokud by však $\beta = 0$, pak by původní klauzule $\alpha \vee \beta$ nebyla splněna a potom

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (0 \vee 0) = 1 \wedge 0 = 0,$$

tzn. ani nová dvojice klauzulí by nebyla splněna, což chceme.

Tím jsme dostali obecný postup pro převod formule z CNF do 3-CNF.

Příklad 2.3.1. Máme formuli $\varphi(a, b, c, d) = a \vee b \vee \neg c \vee \neg b \vee d \vee \neg d$ o jedné klauzuli.

2.4 Problém nezávislé množiny v grafu

2.5 Problém kliky v grafu

2.6 Třídy problémů P a NP

Kapitola 3

Konečné automaty

Kapitola 4

Kryptografie

Kapitola 5

Kompilátory

Kapitola 6

Umělá inteligence